Fault-Tolerant Logical Measurements via Homological Measurement

Benjamin Ide,* Manoj G. Gowda, Priya J. Nadkarni, and Guillaume Dauphinais
Xanadu, Toronto, Ontario M5G 2C8, Canada

(Received 19 November 2024; accepted 5 May 2025; published 10 June 2025)

We introduce homological measurement, a framework for measuring the logical Pauli operators encoded in Calderbank-Shor-Steane stabilizer codes. The framework is based on the algebraic description of such codes as chain complexes. Protocols such as lattice surgery and some of its recent generalizations are shown to be special cases of homological measurement. Using this framework, we develop a specific protocol called edge expanded homological measurement for fault-tolerant measurement of arbitrary logical Pauli operators of general quantum low density parity-check codes, requiring a number of ancillary qubits growing only linearly with the weight of the logical operator measured, and guarantee that the distance of the code is preserved. We further benchmark our protocol numerically in a photonic architecture based on Gottesman-Kitaev-Preskill qubits, showing that the logical error rates of various codes are on par with other methods requiring more ancilla qubits.

DOI: 10.1103/PhysRevX.15.021088 Subject Areas: Quantum Information

I. INTRODUCTION

Because of the fragile nature of quantum systems, one of the most promising approaches to build a useful fault-tolerant large scale quantum computer is to encode the information using stabilizer codes [1,2]. Two-dimensional topological codes [3,4] have long been studied due to their relatively high resilience to noise as evidenced by their high thresholds [5,6] and their planar layout; however, they suffer from a low encoding rate [7]. This contributes to a large qubit overhead requirement in practical applications using these codes [8,9].

More general quantum low density parity-check (qLDPC) codes have recently attracted significant interest [10–31]. While in general they require some degree of long-range connectivity, they do not suffer from the same bounds relating their size, distance, and rate as their local counterparts, allowing for the design of qLDPC codes that require significantly less overhead.

Photonics platforms allow for flexibility of connectivity since components of the quantum processor can be connected via optical fibers [32–35]; thus they are an ideal match for qLDPC codes. The optical nature of the qubits used also makes measurement-based quantum computing [36–40] a natural choice. In some photonic architectures [33,41], the effective noise experienced by a qubit scales with the number and weight of stabilizers in which the qubit participates. Further, the dominant source of noise in this architecture is photon loss, and this physical source of noise works nicely with assumptions of fault tolerance. As such, this work is further motivated by specific needs of this kind of photonics-based architecture to develop fault-tolerant, qLDPC, low space overhead measurement protocols.

A key ingredient to run a useful algorithm in such a platform is the ability to perform logical Pauli measurements. Clifford gates can be implemented using joint logical Pauli measurements alongside the ability to prepare fault-tolerantly encoded Pauli eigenstates [42]. Additionally, having access to some encoded non-Clifford states is sufficient to achieve a universal gate set [43]. While lattice surgery techniques have been widely studied for topological codes [44–47], relatively little work has focused on more general qLDPC codes. Homomorphic logical measurements [48] and generalized lattice surgery [49] both maintain qLDPC properties of the codes they act on, but require a large additional overhead scaling at least quadratically with the distance, and CSS code surgery [50] cannot guarantee fault tolerance, as well as being limited to measuring CSS logical Pauli operators. The overhead of Ref. [49] was improved in Ref. [51], but it is only improved to linear overhead in cases where the logical operators have a certain structure related to edge expansion.

In this work, we introduce the mathematical framework of homological measurement based on the chain complex description of CSS codes. Using this framework, we develop a scheme to measure any multiqubit logical Pauli operator with a number of additional physical qubits scaling linearly with the weight of the logical operator being measured. Given a code with a logical operator to

*Contact author: ben.ide@xanadu.ai

measure, our protocol gives a fault-tolerant method of extending the code while including the logical operator in the stabilizer group.

Homological measurement hinges on the fact that a CSS code $\mathcal{C}$ can naturally be described by a chain complex of length 2, which comprises 3 vector spaces. One of these describes the physical qubits of the system, while the other 2 spaces describe the X and Z syndrome spaces. Linear maps connecting these spaces, called boundary maps, are given by the X- and Z-type check matrices.

Since CSS codes can be described by chain complexes, homological algebra [52] can be used to describe features of CSS codes and operations on CSS codes. For example, logical operators are given by the homology and cohomology groups of the chain complex. One such operation that is used to great effect in this work is the mapping cone applied to maps between chain complexes called chain maps.

The main idea behind homological measurement is to create an ancillary quantum code also described via a chain complex $\mathcal{A}$ and a chain map f from $\mathcal{A}$ to $\mathcal{C}$. A new CSS code is then available via the chain complex resulting from the mapping cone of f. The logical operators in this new code can be studied via the mathematics of the well-established literature on homological algebra [52]. Various known fault-tolerant logical measurement schemes can be described straightforwardly as homological measurements, including lattice surgery [44] and generalized lattice surgery [49,51]. The mapping cone construction has also been used to develop methods to reduce the weight of stabilizer generators of CSS codes [31,53–55]. By carefully choosing the chain complex $\mathcal{A}$ and chain map f, we can engineer a system to perform fault-tolerant logical Pauli operator measurements for arbitrary CSS qLDPC codes with fewer resources than previous methods. Additionally, by exploiting edge expansion of a graph structure implied by the chain complex $\mathcal{A}$, we are able to maintain the distance with much smaller ancillary systems compared to previous methods. We call this edge expanded homological measurement.

A self-contained introduction to the relevant concepts of graph theory and homological algebra is presented in Sec. II, followed by the description of the homological measurement framework as well as our protocol using this framework in Sec. III. Numerical simulations benchmarking our protocol for the measurement-based implementation of various quantum error correcting codes using optical Gottesman-Kitaev-Preskill (GKP) qubits [56] are presented in Sec. IV, followed by concluding remarks in Sec. V.

II. BACKGROUND AND NOTATION

A. Graph theory

The construction in this work relies on the notion of a hypergraph which generalizes graphs so that "edges" are arbitrary subsets of a vertex set.

Definition 1. A hypergraph $\mathcal{G}(V, E)$ is given by a vertex set V and an edge set $E \subseteq \mathcal{P}(V)$ where $\mathcal{P}(V)$ is the power set of V.

Graphs and hypergraphs will mostly be given by incidence matrices.

Definition 2. An edge-vertex incidence matrix M for a (hyper)graph $\mathcal{G}(V, E)$ is a $|E| \times |V|$ matrix where rows represent edges and columns represent vertices given by

$$M_{i,j} = \begin{cases} 1 & v_j \in e_i \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

In this article, when the term *incidence matrix* is used without qualification, we mean the edge-vertex incidence matrix. We allow edges to be repeated, so the edge set E could be a multiset, and correspondingly, the incidence matrix could have multiple identical rows.

We use the following generalization to hypergraphs for the Cheeger constant.

Definition 3. Consider the incidence matrix M of a (hyper)graph $\mathcal{G}(V, E)$. The boundary of $S \subset V$ is

$$\partial S = \{e \in E \,||e \cap S| \text{is odd}\}. \tag{2}$$

The Cheeger constant of $\mathcal{G}$ is given by

$$\begin{aligned} h(\mathcal{G}) &= \min\left\{\frac{|\partial S|}{|S|}\middle| S \subset V, |S| \leq |V|/2\right\} \\ &= \min\left\{\frac{\mathrm{wt}(Mv)}{\mathrm{wt}(v)}\middle| \mathrm{wt}(v) \leq |V|/2\right\}. \end{aligned} \tag{3}$$

This definition matches the usual definition for graphs. Further, if the same matrix M is instead interpreted as a biadjacency matrix for a Tanner graph where rows represent checks and columns represent variables, this matches the Cheeger constant definition in Ref. [51].

Definition 4. If M is an incidence matrix, then a cycle basis is given by the rows of any full-rank matrix N such that $\ker N \cong \mathrm{im} M$. The matrix N can be seen as a cycle-edge incidence matrix where the edge given by the ith row of M is given by the ith column of N, and a row of N defines the edges of a cycle in the cycle basis.

B. Homological algebra

The following is designed to be mostly self-contained, but interested readers should consult standard references to learn more [52,57].

A *chain complex* $\mathcal{C}$ is an ordered sequence of finite dimensional vector spaces $\{C_i\}$ over $\mathbb{F}_2$ with linear transformations between consecutive spaces, $\partial_{i+1}\colon C_{i+1} \to C_i$, such that $\partial_i \circ \partial_{i+1} = 0$:

$$\mathcal{C}\colon \cdots \to C_{i+1} \xrightarrow{\partial_{i+1}} C_i \xrightarrow{\partial_i} C_{i-1} \to \cdots.$$

We are interested only in chain complexes with a finite number of vector spaces, i.e., for indices $m, m+1, \ldots, n$:

$$\mathcal{C}: C_n \xrightarrow{\partial_n} \cdots \xrightarrow{\partial_{m+2}} C_{m+1} \xrightarrow{\partial_{m+1}} C_m.$$

A chain complex with ℓ spaces is called an ℓ-term chain complex. Since $\partial_i \circ \partial_{i+1} = 0$, we have $\mathrm{im}(\partial_{i+1}) \subseteq \ker(\partial_i)$. The ith homology group is defined as $H_i(\mathcal{C}) = \ker(\partial_i)/\mathrm{im}(\partial_{i+1})$. For a chain with a finite number of terms, to calculate the homology groups assume that any unspecified spaces are the trivial space 0 with trivial maps. Note that throughout this work, the symbol H is also used for parity-check and stabilizer matrices, but the usage is clear from context. The dual of a chain complex $\mathcal{C}$ is a *cochain complex* $\mathcal{C}^*$:

$$\mathcal{C}^*: \; \cdots \leftarrow C^*_{i+1} \xleftarrow{\delta_i} C^*_i \xleftarrow{\delta_{i-1}} C^*_{i-1} \leftarrow \cdots,$$

where C^*_i is the space of all linear functionals on C_i. Note that $C^*_i \cong C_i$ since we consider only finite dimensional vector spaces. The corresponding dual of the homology groups are the cohomology groups $H^i(\mathcal{C}) = \ker \delta_i / \mathrm{im}\delta_{i-1}$. We assume that vector spaces and their duals are equipped with the standard basis so that, when viewed as matrices, $\delta^{\mathrm{T}}_{i-1} = \partial_i$.

A chain is said to be exact at C_i if $\mathrm{im}(\partial_{i+1}) = \ker(\partial_i)$ and is said to be an exact sequence if it is exact at each C_i. An exact sequence of the form $0 \to X \to Y \to Z \to 0$ is called a short exact sequences. Any exact sequence with more than three nontrivial spaces is called a long exact sequence.

Suppose that $0 \to V_1 \to \cdots \to V_r \to 0$ is an exact sequence of finite dimensional vector spaces. Then

$$\sum_{i=1}^{r} (-1)^i \dim V_i = 0. \tag{4}$$

This is a generalization of the rank-nullity theorem.

Consider two chain complexes $\mathcal{A}$ and $\mathcal{C}$ and maps $f_i\colon A_i \to C_i$ linking them:

$$\begin{tikzcd}
\mathcal{A}: \arrow[d, "f"] & \cdots \arrow[r] & A_i \arrow[r, "\partial_i^{\mathcal{A}}"] \arrow[d, "f_i"] & A_{i-1} \arrow[r] \arrow[d, "f_{i-1}"] & \cdots \\
\mathcal{C}: & \cdots \arrow[r] & C_i \arrow[r, "\partial_i^{\mathcal{C}}"] & C_{i-1} \arrow[r] & \cdots
\end{tikzcd} \tag{5}$$

When $\partial_i^{\mathcal{C}}(f_i(a)) = f_{i-1}(\partial_i^{\mathcal{A}}(a))$ for all $a \in A_i$, the maps f_i are collectively called a chain map $f\colon \mathcal{A} \to \mathcal{C}$. This condition is often written more compactly as $\partial^{\mathcal{C}} f = f \partial^{\mathcal{A}}$. The mapping cone of f is defined to be the chain complex with spaces $\mathrm{cone}(f)_i = C_i \oplus A_{i-1}$ and maps $\partial_i^{\mathrm{cone}(f)}\colon \mathrm{cone}(f)_i \to \mathrm{cone}(f)_{i-1}$:

$$\begin{tikzcd}
A_{i-1} \arrow[r, "\partial^{\mathcal{A}}_{i-1}"] \arrow[dr, "f_{i-1}"] & A_{i-2} \\
\oplus & \oplus \\
C_i \arrow[r, "\partial_i^{\mathcal{C}}"'] & C_{i-1}
\end{tikzcd}$$

$$\mathrm{cone}(f)_i \xrightarrow{\partial_i^{\mathrm{cone}(f)}} \mathrm{cone}(f)_{i-1}$$

$$\partial_i^{\mathrm{cone}(f)} = \begin{array}{c c} & \begin{array}{cc} C_i & A_{i-1} \end{array} \\ \begin{array}{c} C_{i-1} \\ A_{i-2} \end{array} & \begin{pmatrix} \partial_i^{\mathcal{C}} & -f_{i-1} \\ 0 & -\partial^{\mathcal{A}}_{i-1} \end{pmatrix} \end{array}. \tag{6}$$

In this paper we restrict to $\mathbb{F}_2$, so the minus signs in the map above will be omitted.

The mapping cone gives the short exact sequence,

$$0 \to C_k \xrightarrow{\iota} \mathrm{cone}(f)_k \xrightarrow{\pi} A_{k-1} \to 0, \tag{7}$$

for all indices k where ι is inclusion and π is projection. Applying the snake lemma [52,57] with the chain map f as the connecting map, we obtain the following long exact sequence on homology where f_*, ι_*, and π_* are maps derived from f, ι, and π:

$$\begin{tikzcd}
\cdots \arrow[r, "\iota_*"] & H_{k+1}(\mathrm{cone}(f)) \arrow[r, "\pi_*"] & H_k(\mathcal{A}) \arrow[dll, "f_*"'] \\
H_k(\mathcal{C}) \arrow[r, "\iota_*"'] & H_k(\mathrm{cone}(f)) \arrow[r, "\pi_*"'] & \cdots
\end{tikzcd} \tag{8}$$

This exact sequence allows us to study the logical operators that remain after logical measurement is conducted via the mapping cone. Interested readers should consult Ref. [52], Chap. 1 up to Lemma 1.5.3, for a full treatment of this result.

C. Codes as chain complexes

There is a natural correspondence between codes and chain complexes. Let $H \in \mathbb{F}_2^{n-k \times n}$ be a parity-check matrix of an $[n, k, d]$ (classical) linear code, where n is the number of variables, k is the dimension, and d is the distance. Then we can express this code as

$$\mathbb{F}_2^n \xrightarrow{H} \mathbb{F}_2^{n-k}, \tag{9}$$

which vacuously satisfies the definition of a chain complex. Conversely, any chain complex with only one map gives a classical linear code.

Now consider an $[[n, k, d]]$ CSS code. Define d_X to be the minimum weight of nontrivial X-logical operators and d_Z to be the minimum weight of nontrivial Z-logical operators. Then n is the number of physical qubits, k is the number of logical qubits, and $d = \min\{d_X, d_Z\}$ is the distance. A CSS code $\mathrm{CSS}(H_X, H_Z)$ is generated by n_X X-stabilizer generators, not necessarily independent, given by rows of the matrix H_X and n_Z Z-stabilizer generators given by rows of H_Z. Since $H_Z H_X^{\mathrm{T}} = 0$, we can treat this as the chain complex,

$$\mathbb{F}_2^{n_X} \xrightarrow{H_X^{\mathrm{T}}} \mathbb{F}_2^{n} \xrightarrow{H_Z} \mathbb{F}_2^{n_Z}, \tag{10}$$

with the qubits in the center. Conversely, we can derive a CSS code from any two consecutive boundary maps ∂_{i+1} and ∂_i by setting $H_X^{\mathrm{T}} = \partial_{i+1}$ and $H_Z = \partial_i$. The X-logical operators commute with the Z stabilizers (i.e., they are in $\ker H_Z$) and are not X stabilizers (i.e., they are not in $\mathrm{im} H_X^{\mathrm{T}}$), which make them nontrivial elements of the first homology group H_1. The dual gives the cochain

$$\mathbb{F}_2^{n_X} \xleftarrow{H_X} \mathbb{F}_2^{n} \xleftarrow{H_Z^{\mathrm{T}}} \mathbb{F}_2^{n_Z}, \tag{11}$$

which implies that the Z logicals are given by the first cohomology group H^1. Since these descriptions of the CSS code are duals of each other, one can also view Eq. (11) as the chain with Eq. (10) as its cochain, which is common in the literature. We choose to consider Eq. (10) as the chain complex because this is natural for measurement of X-logical operators.

Since CSS codes and 3-term chain complexes are equivalent, we can create new codes using mapping cones. Consider a chain map $f: \mathcal{A} \to \mathcal{C}$, where $\mathcal{C}$ is the chain complex $C_2 \xrightarrow{H_X^{\mathrm{T}}} C_1 \xrightarrow{H_Z} C_0$ given by a CSS code and $\mathcal{A}$ is a chain complex $A_1 \xrightarrow{\partial_1} A_0 \xrightarrow{\partial_0} A_{-1}$. The cone code is given by

$$\begin{tikzcd} A_1 \arrow[r, "\partial_1"] \arrow[dr, "f_1"] & A_0 \arrow[r, "\partial_0"] \arrow[dr, "f_0"] & A_{-1} \\ \oplus & \oplus & \oplus \\ C_2 \arrow[r, "H_X^{\mathrm{T}}"'] & C_1 \arrow[r, "H_Z"'] & C_0 \end{tikzcd} \tag{12}$$

which has stabilizer matrices

$$\tilde{H}_X = \begin{pmatrix} H_X & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} \end{pmatrix}, \qquad \tilde{H}_Z = \begin{pmatrix} H_Z & f_0 \\ & \partial_0 \end{pmatrix}. \tag{13}$$

III. HOMOLOGICAL MEASUREMENT

The goal of this work is to measure logical operators fault tolerantly while minimizing the number of qubits required. Naively, one would simply measure the desired logical operator directly, removing it from the logical group and adding it to the stabilizer group. However, for codes with large distances, this would result in measuring directly a high-weight operator, a process which is prone to errors. Consequently, protocols have been developed [44,49,51] to infer the value of these higher-weight logical operators by measuring a number of lower-weight operators instead. To accomplish this, one must modify the original code(s) in a way that leaves the logical information left unaffected by the desired measurement. This can be done for any CSS code using the generalized lattice surgery protocol from Ref. [49]; however, the resource overhead is prohibitively large. The situation is improved in Ref. [51] by exploiting edge expansion that is present in the Tanner graph, but except in some specific cases where the edge expansion is sufficient, this still uses more resources than necessary. The protocol developed in this work uses fewer resources in general by improving the edge expansion directly rather than accepting it as a fixed quantity in a process we call edge expanded homological measurement.

Given a CSS code $\mathcal{C}$ and a logical operator of $\mathcal{C}$, we design an ancillary code $\mathcal{A}$ and a chain map $f: \mathcal{A} \to \mathcal{C}$ such that this logical operator is in the stabilizer group of the cone code of f. We require that the result of the logical measurement can be inferred correctly despite the presence of errors, and as such, the distance of the cone code should be at least that of $\mathcal{C}$ and stabilizer weights should be reasonably low. Finally, no other logical operators of $\mathcal{C}$ should be measured so that the remaining logical information is left intact.

A discussion of the properties of cone codes in general is given in Sec. III A, along with examples of past measurement protocols that fit the cone code framework. The edge expanded homological measurement construction for measuring any X- or Z-logical operator is given in Sec. III B, and a protocol for using this construction in a measurement scheme is given in Sec. III C. A construction for measuring operators built from a mixture of logical X, Y, and Z Pauli operators is given in Sec. III D.

A. Mathematical properties of cone codes

Consider a CSS code given by stabilizer matrices H_X and H_Z written as the chain complex,

$$\mathcal{C}: C_2 \xrightarrow{H_X^{\mathrm{T}}} C_1 \xrightarrow{H_Z} C_0, \tag{14}$$

an ancillary CSS code with X-stabilizer matrix ∂_1^{T} and Z-stabilizer matrix ∂_0 written as the chain complex,

$$\mathcal{A}: A_1 \xrightarrow{\partial_1} A_0 \xrightarrow{\partial_0} A_{-1}, \tag{15}$$

and a chain map $f: \mathcal{A} \to \mathcal{C}$. We will use the notation

$$\mathrm{cone}(f) = \tilde{\mathcal{C}}: \tilde{C}_2 \xrightarrow{\tilde{H}_X^{\mathrm{T}}} \tilde{C}_1 \xrightarrow{\tilde{H}_Z} \tilde{C}_0 \tag{16}$$

for the cone code of f.

Before constructing a specific $\mathcal{A}$ and f, we first make a few general observations. We choose to interpret the resulting cone code as a subsystem code, and so $\dim H_1(\tilde{\mathcal{C}}) = \tilde{k} + \tilde{r}$, where $\tilde{k}$ is the number of logical qubits and $\tilde{r}$ is the number of gauge qubits. Applying the generalization of the rank-nullity theorem given in Eq. (4) to the long exact sequence of Eq. (8) for $\tilde{\mathcal{C}}$, and treating any of the nontrivial operators of the cone code that arise from $\mathcal{A}$ as gauges, we have the following:

$$\tilde{k} = k + (\dim\ker \tilde{H}_X^{\mathrm{T}} - \dim\ker H_X^{\mathrm{T}}) - \dim\ker \partial_1 \tag{17}$$

$$\tilde{r} = \dim(\ker \partial_0/\mathrm{im}\,\partial_1) + (\dim\ker \tilde{H}_Z^{\mathrm{T}} - \dim\ker \partial_0^{\mathrm{T}}) - \dim\ker H_Z^{\mathrm{T}}. \tag{18}$$

The term $(\dim\ker \tilde{H}_X^{\mathrm{T}} - \dim\ker H_X^{\mathrm{T}})$ measures the amount of redundancy of the X-stabilizer generators in $\tilde{\mathcal{C}}$ that were not directly copied from $\mathcal{C}$. Lemma 1 below shows that this remaining redundancy is isomorphic to a subspace of $\ker\partial_1$ and that the remaining elements of $\ker\partial_1$ contribute to reduction in $\tilde{k}$. Equation (18) mirrors Eq. (17) with the roles of $\mathcal{A}$ and $\mathcal{C}$ swapped and the roles of X and Z swapped, noting in particular that the number of logical qubits in $\mathcal{A}$ is $\dim(\ker \partial_0/\mathrm{im}\,\partial_1)$. When applying a cone code construction to measure logical operators, as it is used in Sec. III B, the reduction of $\tilde{k}$ corresponds to the measurement of a logical operator.

Lemma 1 (Characterization of $\ker\partial_1$*).* Given a CSS code $\mathcal{C} = \mathrm{CSS}(H_X, H_Z)$ interpreted as a chain complex $\mathcal{C}: C_2 \xrightarrow{H_X^{\mathrm{T}}} C_1 \xrightarrow{H_Z} C_0$, a chain complex $\mathcal{A}: A_1 \xrightarrow{\partial_1} A_0 \xrightarrow{\partial_0} A_{-1}$, and a chain map $f: \mathcal{A} \to \mathcal{C}$,

$$\ker\ \partial_1 \cong \mathcal{S} \oplus \mathcal{L}, \tag{19}$$

where $\mathcal{S}$ is the space of X stabilizers of $\mathcal{C}$ that can be measured via the new stabilizer generators of $\tilde{H}_X$ and $\mathcal{L}$ is the space of nontrivial X-logical operators of $\mathcal{C}$ that can be measured via the new stabilizer generators of $\tilde{H}_X$.

Proof. Recall that the mapping cone of f gives a CSS code $\tilde{\mathcal{C}}$ with stabilizer matrices:

$$\tilde{H}_X = \begin{pmatrix} H_X & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} \end{pmatrix}, \qquad \tilde{H}_Z = \begin{pmatrix} H_Z & f_0 \\ & \partial_0 \end{pmatrix}. \tag{20}$$

Let $\vec{v} \in \ker\partial_1$ and define

$$S_{\vec{v}} = \left(\begin{pmatrix} f_1 \\ \partial_1 \end{pmatrix} \vec{v} \right)^{\mathrm{T}}. \tag{21}$$

By construction, $S_{\vec{v}}$ is a sum of some of the new X-type stabilizer generators of $\tilde{\mathcal{C}}$ and therefore is an X stabilizer of $\tilde{\mathcal{C}}$. Since $\vec{v} \in \ker\partial_1$, $S_{\vec{v}}$ has no support on the ancillary qubits.

Since $\tilde{H}_Z$ restricted to the original qubits of $\mathcal{C}$ is simply H_Z, we know that $S_{\vec{v}}|_{\mathcal{C}}$ represents an X operator that commutes with H_Z. Thus, $S_{\vec{v}}$ is either an X stabilizer or a nontrivial X logical of $\mathcal{C}$. ∎

One use of Lemma 1 is to show that our protocol does not inadvertently measure extra logical operators by enforcing $\dim\ker\partial_1 = 1$.

Even with this generality the Z distance of $\tilde{\mathcal{C}}$ is maintained.

Theorem 1. (Z distance). If the nontrivial operators of the cone code in Eq. (13) resulting from the logicals of the ancilla code are treated as gauges as in Eqs. (17) and (18), then the dressed Z distance $\tilde{d}_Z$ is maintained, i.e., $\tilde{d}_Z \geq d_Z$.

Proof. The dressed Z distance is the same as the Z distance of the stabilizer code where a full basis of the Z gauges are included as stabilizers. To that end, since all gauges arise from logicals of the ancilla code, replace ∂_0 with a matrix G such that $\ker G \cong \mathrm{im}\partial_1$ leaving us to study the following stabilizer code:

$$\tilde{H}_X = \begin{pmatrix} H_X & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} \end{pmatrix}, \qquad \tilde{H}'_Z = \begin{pmatrix} H_Z & f_0 \\ & G \end{pmatrix}. \tag{22}$$

Let $v = \binom{v_C}{v_A} \in \ker\tilde{H}_X$ where v_C is defined on the block of qubits from $\mathcal{C}$ and v_A is on the block of qubits from $\mathcal{A}$. This implies

$$H_X v_C = 0 \tag{23}$$

and

$$f_1^{\mathrm{T}} v_C + \partial_1^{\mathrm{T}} v_A = 0. \tag{24}$$

From Eq. (23), if v_C^{T} is not a Z stabilizer of $\mathcal{C}$, then it must be a logical of $\mathcal{C}$, and so $\mathrm{wt}(v) \geq \mathrm{wt}(v_C) \geq d_Z$. Suppose that v_C^{T} is a Z stabilizer of the original code; therefore there is a vector w_1 such that $v_C = H_Z^{\mathrm{T}} w_1$. Since f is a chain map, we know that $H_Z f_1 = f_0 \partial_1$, or equivalently, $f_1^{\mathrm{T}} H_Z^{\mathrm{T}} = \partial_1^{\mathrm{T}} f_0^{\mathrm{T}}$. Using this,

$$f_1^{\mathrm{T}} v_C = f_1^{\mathrm{T}} H_Z^{\mathrm{T}} w_1 = \partial_1^{\mathrm{T}} f_0^{\mathrm{T}} w_1, \tag{25}$$

and so from Eq. (24) we find

$$\partial_1^{\mathrm{T}}(f_0^{\mathrm{T}} w_1 + v_A) = 0, \tag{26}$$

and since $\ker G \cong \mathrm{im}\partial_1$, we also have $\mathrm{im} G^{\mathrm{T}} \cong \ker\partial_1^{\mathrm{T}}$, so there exists a w_2 such that $G^{\mathrm{T}} w_2 = f_0^{\mathrm{T}} w_1 + v_A$. Now we have

$$\begin{aligned} \tilde{H}_Z'^{\mathrm{T}} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} &= \begin{pmatrix} H_Z^{\mathrm{T}} w_1 \\ f_0^{\mathrm{T}} w_1 + G^{\mathrm{T}} w_2 \end{pmatrix} \\ &= \begin{pmatrix} v_C \\ f_0^{\mathrm{T}} w_1 + f_0^{\mathrm{T}} w_1 + v_A \end{pmatrix} = \begin{pmatrix} v_C \\ v_A \end{pmatrix} = v; \end{aligned} \tag{27}$$

therefore, in the case where v_C^{T} is a Z stabilizer of the original code, v^{T} is a stabilizer of the code in Eq. (22).

Thus, the Z distance for Eq. (22) is at least d_Z, and therefore the dressed Z distance of $\tilde{\mathcal{C}}$ is also at least d_Z. ■

This framework is sufficiently general to encompass many known schemes, including both lattice surgery [44] and generalizations of lattice surgery [49,51] as we show in the following examples.

Example 1 (Lattice surgery of surface codes [44]). Consider two surface codes of equal X distance d, $\mathcal{C}_1 = \mathrm{CSS}(H_{X,1}, H_{Z,1})$ and $\mathcal{C}_2 = \mathrm{CSS}(H_{X,2}, H_{Z,2})$. Let $\bar{X}_1$ be a weight d logical operator of $\mathcal{C}_1$ and $\bar{X}_2$ be a weight d logical operator of $\mathcal{C}_2$ such that $\mathcal{Q}_1 = \mathrm{supp}\bar{X}_1 = \{q_{1,1}, q_{1,2}, \ldots, q_{1,d}\}$, and $\mathcal{Q}_2 = \mathrm{supp}\bar{X}_2 = \{q_{2,1}, q_{2,2}, \ldots, q_{2,d}\}$ and suppose we wish to measure $\bar{X}_1\bar{X}_2$.

Define the chain complex $\mathcal{C}$ in Eq. (14) by $H_X = H_{X,1} \oplus H_{X,2}$ and $H_Z = H_{Z,1} \oplus H_{Z,2}$. Define $H_Z|^{\mathrm{nz}}_{\mathcal{Q}_1}$ to be H_Z restricted to the columns given by $\mathcal{Q}_1$ with zero rows removed ("nz" refers to no zeros), and similarly for $\mathcal{Q}_2$. Define the chain $\mathcal{A}$ by the maps

$$\partial_1 = \begin{pmatrix} 1 & 1 & & & \\ & 1 & 1 & & \\ & & \ddots & \ddots & \\ & & & 1 & 1 \end{pmatrix}_{(d-1)\times d}, \qquad \partial_0 = 0, \tag{28}$$

where ∂_1 is equal to both $H_Z|^{\mathrm{nz}}_{\mathcal{Q}_1}$ and $H_Z|^{\mathrm{nz}}_{\mathcal{Q}_2}$ up to permutations, and define the chain map $f\colon \mathcal{A} \to \mathcal{C}$ by

$$(f_1)_{i,j} = \delta_{i,q_{1,j}} + \delta_{i,q_{2,j}}, \tag{29}$$

and

$$(f_0)_{i,j} = \begin{cases} 1 & \text{if } \begin{cases} (H_Z)_{i,q_{1,j}} = (H_Z)_{i,q_{1,j+1}} = 1 \text{ or} \\ (H_Z)_{i,q_{2,j}} = (H_Z)_{i,q_{2,j+1}} = 1 \end{cases} \\ 0 & \text{otherwise,} \end{cases} \tag{30}$$

where f_1 has dimensions $n \times d$, f_0 has dimensions $n_Z \times (d-1)$. The cone code of f gives the merged system after performing lattice surgery.

Note that f_0 embeds the rows of ∂_1 into the corresponding rows of H_Z, and f_1^{T} has d rows of weight two which sum to $\bar{X}_1\bar{X}_2$. From the structure of f_1 and the fact that ∂_1 has even weight rows, we see that $\bar{X}_1\bar{X}_2$ is a stabilizer of the resulting code and can therefore be measured.

Example 2 (Generalized lattice surgery [49]). Consider measuring an X-logical operator $\bar{X}$ of weight w of the CSS code $\mathcal{C} = \mathrm{CSS}(H_X, H_Z)$. Let $\mathcal{Q} = \{q_1, \ldots, q_w\} = \mathrm{supp}\bar{X}$, $H_Z|_{\mathcal{Q}}$ be the matrix H_Z restricted to the qubits in $\mathcal{Q}$, and $H_Z|^{\mathrm{nz}}_{\mathcal{Q}}$ be the restriction to nonzero rows of $H_Z|_{\mathcal{Q}}$. Consider the following parity-check matrix of dimensions $(r-1)\times r$ for the $[r,1,r]$ classical repetition code:

$$H_r = \begin{pmatrix} 1 & 1 & & & \\ & 1 & 1 & & \\ & & \ddots & \ddots & \\ & & & 1 & 1 \end{pmatrix}, \tag{31}$$

where the choice $r = d$ with d the distance of $\mathcal{C}$ is used in general to ensure that distance is maintained [49]. The ancillary chain $\mathcal{A}$ is given by the hypergraph product code [58],

$$\partial_1^{\mathrm{T}} = \begin{pmatrix} I_r \otimes H_Z|^{\mathrm{nz}\,\mathrm{T}}_{\mathcal{Q}} & H_r^{\mathrm{T}} \otimes I_w \end{pmatrix},$$
$$\partial_0 = \begin{pmatrix} H_r \otimes I_{n'_Z} & I_{r-1} \otimes \partial_1 \end{pmatrix}, \tag{32}$$

where n'_Z is the number of rows of $H_Z|^{\mathrm{nz}}_{\mathcal{Q}}$.

Let h_j be the index of the jth nonzero row of $H_Z|_{\mathcal{Q}}$, and we define the chain map $f\colon \mathcal{A} \to \mathcal{C}$ by

$$(f_1)_{i,j} = \begin{cases} \delta_{i,q_j} & j \le w \\ 0 & \text{otherwise} \end{cases} \tag{33}$$

and

$$(f_0)_{i,j} = \begin{cases} \delta_{i,h_j} & j \le n'_Z \\ 0 & \text{otherwise,} \end{cases} \tag{34}$$

where f_1 is an $n \times rw$ matrix and f_0 is an $n_Z \times [rn'_Z + (r-1)w]$ matrix. The cone code Eq. (13) then gives the generalized lattice surgery construction for measurement of $\bar{X}$.

Example 3 (Generalized lattice surgery with reduced r [51]). The choice of $r = d$ in Example 2 is not necessarily optimal, but due to the potential presence of gauges, it is difficult to find an analysis of when a reduced choice of r is admissible. To address this, Ref. [51] includes all Z-logical operators of the ancilla system from Example 2 as stabilizers in ∂_0 so that the resulting cone code has no gauges. It is then proven [following a similar result in weight reduction, Ref. [54] Lemma 8(8)] that the edge expansion of the Tanner graph of H_Z restricted to the variable nodes Q can be used to find an upper bound on r that will maintain distance.

The new rows of ∂_0 in this scheme are not necessarily low weight, and indeed there does not always exist a choice that could be considered LDPC. For example, the measurement of a weight d logical operator of a distance d toric code would give a row in ∂_0 of weight d. This shortcoming is addressed in our scheme.

Note that in Ref. [51], a parameter L is used instead of r with the relationship $2r - 1 = L$ for odd L. Even values of L give a slightly different construction that, while still falling neatly within the cone code framework, does not provide a useful measurement technique and will be addressed in the Appendix.

Examples 2 and 3 (after Refs. [49,51]) along with another recent work [59] all have similar schemes for measuring individual logicals, but note that joint measurements are conducted differently across these three works. These examples show that the homological measurement framework elegantly includes many past schemes. In the remainder of the section, we work within the cone code framework to develop a more optimized measurement scheme.

B. Measuring an X-logical operator

Let $\mathcal{C} = \mathrm{CSS}(H_X, H_Z)$ be an $[\![n, k, d = \min\{d_X, d_Z\}]\!]$ CSS code and w_X and q_X (w_Z and q_Z) be the maximum row and column weights of H_X (H_Z). Given an X-logical operator $\bar{X}$ to measure, define $\mathcal{Q} = \mathrm{supp}\bar{X} = \{q_1, \ldots, q_w\}$, where $w = |\mathcal{Q}|$ is the weight of $\bar{X}$ and $q_i < q_{i+1}$ for all i so that the qubit labels are ordered. This works identically for Z-logical operators by swapping the roles of X and Z. To perform a joint measurement across multiple codes $\mathcal{C}_i = \mathrm{CSS}(H_{X,i}, H_{Z,i})$, let $\mathcal{C} = \mathrm{CSS}(\bigoplus_i H_{X,i}, \bigoplus_i H_{Z,i})$ and proceed as usual.

We begin by choosing f_1 to be the $n \times w$ matrix:

$$(f_1)_{i,j} = \delta_{i,q_j}. \tag{35}$$

If we choose a ∂_1 with even weight rows, then the sum of the new X stabilizers given by the rows $(f_1^{\mathrm{T}}\, \partial_1^{\mathrm{T}})$ from Eq. (13) is $(\bar{X}\, 0)$ as desired. This choice of f_1 requires f_0 and ∂_1 to be chosen such that

$$f_0 \partial_1 = H_Z f_1 = H_Z|_{\mathcal{Q}}, \tag{36}$$

so that f is a valid chain map [equivalently, to enforce commutativity of the stabilizer matrices in Eq. (13)]. The notation $H_Z|_{\mathcal{Q}}$ means H_Z restricted to the columns given by the qubits Q. From Eq. (36), we see that all rows of $H_Z|_{\mathcal{Q}}$ must be in the row space of ∂_1, and in order to have a sparse choice of f_0, it should require very few rows of ∂_1 to generate rows of $H_Z|_{\mathcal{Q}}$, and rows of ∂_1 should not be reused many times to represent different rows of $H_Z|_{\mathcal{Q}}$. We will construct a stabilizer code which often requires a nonzero choice for ∂_0 to ensure that there are no gauges, so the sparsity of ∂_0 must also be considered. Given this setup, the following Theorem 2 gives a sufficient condition to ensure that the X distance is maintained in the cone code.

Theorem 2 (X distance). Given a CSS code defined by $\mathcal{C}: C_2 \xrightarrow{H_X^{\mathrm{T}}} C_1 \xrightarrow{H_Z} C_0$, a logical operator $\bar{X}$ we wish to measure from $\mathcal{C}$, a chain complex $\mathcal{A}: A_1 \xrightarrow{\partial_1} A_0 \xrightarrow{\partial_0} A_{-1}$ such that $\ker \partial_0 \cong \mathrm{im}\partial_1$, and a chain map $f: \mathcal{A} \to \mathcal{C}$ with f_1 as in Eq. (35), the cone code of f maintains the X distance of $\mathcal{C}$ when the Cheeger constant of the (hyper)graph defined by interpreting ∂_1 as an edge-vertex incidence matrix is at least 1.

Proof. Since no gauges are created and the original X-logical operators are still X-logical operators, we must consider the weight of an unmeasured logical operator $\bar{X}_2$ from the original code, say of weight w_2, multiplied by some collection of the new stabilizer generators from the rows of $(f_1^{\mathrm{T}}\, \partial_1^{\mathrm{T}})$. Label the corresponding w stabilizers $S_{q_1}, \ldots, S_{q_w}$ where S_{q_i} has support in the qubits of the original code on only q_i. To ensure the X distance is maintained, it is sufficient to show that for any subset $R \subseteq \mathcal{Q}$,

$$\mathrm{wt}\left(\bar{X}_2 \prod_{q \in R} S_q\right) \geq d. \tag{37}$$

Fix a subset $R \subseteq \mathcal{Q}$ and define the logical representative $L = \bar{X}_2 \prod_{q \in R} S_q$. Let $R_{\mathrm{int}} = R \cap \mathrm{supp}\bar{X}_2$ and $R_{\mathrm{int}}^c = R \backslash \mathrm{supp}\bar{X}_2$. Note that R is the disjoint union of R_{int} and R_{int}^c. Suppose the size of the overlap between $\bar{X}$ and $\bar{X}_2$ is $i = |\mathrm{supp}\bar{X} \cap \mathrm{supp}\bar{X}_2|$. Because the weight of $\bar{X}\bar{X}_2$ must be at least d, we have

$$w + w_2 - 2i \geq d \Rightarrow |R_{\mathrm{int}}| \leq i \leq \frac{w + w_2 - d}{2} \tag{38}$$

$$\Rightarrow 2|R_{\mathrm{int}}| \leq w + w_2 - d. \tag{39}$$

Let h be the Cheeger constant of the graph with w vertices defined by the incidence matrix ∂_1, recalling that $h \geq 1$ by supposition. The weight of L on the qubits of $\mathcal{C}$ is $w_2 + |R_{\mathrm{int}}^c| - |R_{\mathrm{int}}|$ by definition, and on the qubits of $\mathcal{A}$ it is lower bounded by $h \min\{|R|, w - |R|\}$ by the definition of the Cheeger constant. If $|R| \leq w/2$, then

$$\mathrm{wt}(L) \geq w_2 + |R_{\mathrm{int}}^c| - |R_{\mathrm{int}}| + |R|h \tag{40}$$

$$\geq w_2 + |R_{\mathrm{int}}^c| - |R_{\mathrm{int}}| + |R| \tag{41}$$

$$\geq w_2 \geq d, \tag{42}$$

and if $|R| > w/2$, then noting that $|R_{\mathrm{int}}^c| - |R| = -|R_{\mathrm{int}}|$, we have

$$\mathrm{wt}(L) \geq w_2 + |R_{\mathrm{int}}^c| - |R_{\mathrm{int}}| + (w - |R|)h \tag{43}$$

$$\geq w_2 + |R_{\mathrm{int}}^c| - |R_{\mathrm{int}}| + w - |R| \tag{44}$$

$$= w + w_2 - 2|R_{\mathrm{int}}| \tag{45}$$

$$\geq w + w_2 - (w + w_2 - d) = d. \tag{46}$$

And so, we find that $\mathrm{wt}(L) \geq d$, and thus the X distance of the cone code of f is at least d. ∎

Following choices made in Refs. [49,51,54,59], we start by defining the matrix ∂_1^* to be the $n_Z' \times w$ matrix obtained by restricting H_Z to the columns $\mathcal{Q}$ followed by removing zero rows where n_Z' is the number of nonzero rows:

$$\partial_1^* = H_Z|_{\mathcal{Q}}^{\mathrm{nz}}. \tag{47}$$

The superscript asterisk is to emphasize that this will not be our final choice, but is instead a step toward that choice. Let h_j be the row index of the jth nonzero row of $H_Z|_{\mathcal{Q}}$ and define the $n_Z \times n_Z'$ matrix f_0^* by

$$(f_0^*)_{i,j} = \delta_{i,h_j}. \tag{48}$$

Remark 1. If we were to use f_0^* and ∂_1^* as in Eqs. (47) and (48) and choose $\partial_0 = 0$, the cone code would give us the $r = 1$ construction of Ref. [49]. See also Example 2.

Remark 2. An alternative way to obtain the same maps is to choose $\mathcal{Q}$ as above, and consider the following operations:

$$\begin{array}{c} \begin{pmatrix} H_Z & I_{n_Z} \\ I_n & \end{pmatrix} \\ \Big\downarrow \text{delete columns } \{1,\ldots,n\}\backslash\mathcal{Q} \\ \begin{pmatrix} H_Z|_{\mathcal{Q}} & I_{n_Z} \\ f_1 & \end{pmatrix} \\ \Big\downarrow \text{delete zero rows of } H_Z|_{\mathcal{Q}} \\ \begin{pmatrix} \partial_1^* & f_0^{*\mathrm{T}} \\ f_1 & \end{pmatrix} \end{array} \tag{49}$$

where we delete the columns of $\binom{H_Z}{I_n}$ corresponding to qubits $\{1,\ldots,n\}\backslash\mathcal{Q}$ followed by deleting the rows of $(H_Z|_{\mathcal{Q}}\ I_{n_Z})$ whose corresponding rows of $H_Z|_{\mathcal{Q}}$ are 0.

Note that since $\bar{X}$ must commute with Z stabilizers, rows of ∂_1^* must have even weight. All modifications we make to ∂_1^* to obtain the final ∂_1 will retain this feature so that the sum of the rows $(f_1^{\mathrm{T}}\,\partial_1^{\mathrm{T}})$ gives $(\bar{X}\,0)$. Further, f_0^* is sparse and the row and column weights of ∂_1^* are bounded by w_Z and q_Z. The remaining design problems are finding a sparse choice of ∂_0 and modifying ∂_1^* so that the graph it defines has a Cheeger constant of 1 so that the X distance is maintained. To solve the former problem, we will use a combination of replacing hyperedges with usual edges followed by cellulation. The latter problem is solved via application of Algorithm 1 where new edges of the graph correspond to new rows of ∂_1^* along with corresponding new zero columns in f_0^*.

Remark 3. A positive Cheeger constant implies that there is only a single connected component, which necessarily gives $\dim\ker\partial_1 = 1$, so by Lemma 1, only the desired operator is measured. Therefore joint measurements do not require any extra work after applying Algorithm 1.

Algorithm 1. Greedy algorithm to add edges to a graph to obtain a Cheeger constant of one.

Input: Hypergraph $A = \mathcal{G}(V, E)$ with Cheeger constant $h(A) < 1$.
Output: A hypergraph B with Cheeger constant $h(B) = 1$, the same vertices as A, and a superset of the edges of A.

$E^* \leftarrow E$ and $B \leftarrow \mathcal{G}(V, E^*)$
while $h(B) < 1$ **do**
  `// Find the sparsest cut:`
  $S \leftarrow \mathrm{argmin}_{S\subset V, |S|\leq|V|/2}\{\frac{|\partial S|}{|S|}\}$ where ∂S is calculated using the edges E^*.
  `// Add an appropriate edge:`
  $h^* \leftarrow -\infty$ and initialize an edge e that will be overwritten.
  for v_1 *a vertex of minimum degree in* S **do**
    for v_2 *a vertex of minimum degree in* $V\backslash S$ **do**
      if $h(\mathcal{G}(V, E^* \cup \{(v_1, v_2)\})) > h^*$ **then**
        $h^* \leftarrow h(\mathcal{G}(V, E^* \cup \{(v_1, v_2)\}))$
        $e \leftarrow (v_1, v_2)$
      end
    end
  end
  $E^* \leftarrow E^* \cup \{e\}$ and $B \leftarrow \mathcal{G}(V, E^*)$
end
return B

Example 4. Consider a graph given by vertices,

$$V = \{v_1, \ldots, v_6\},$$

and edges,

$$E = \{(v_1, v_2), (v_2, v_3), (v_4, v_5), (v_5, v_6)\},$$

which can be seen in Fig. 1(a). This can be obtained from ∂_1^* in Eq. (47) when considering measurement of the product of the X-logical operators across two distance 3 surface codes. Applying Algorithm 1, three edges are added. This can be seen in Figs. 1(b)–1(d).

For concreteness, we show the incidence matrix ∂_1 at each stage with a superscript representing which panel of Fig. 1 is described by the matrix. The labeling of the vertices and edges is consistent with the order of rows and columns. For the original graph as shown in Fig. 1(a),

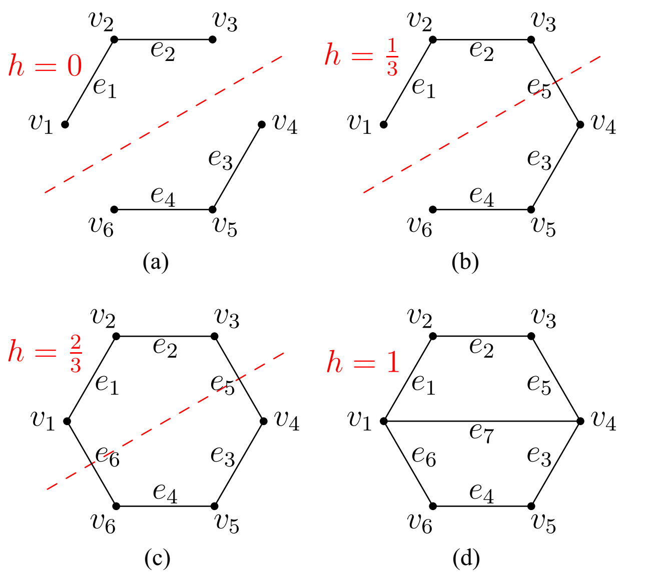

FIG. 1. Visualization of the application of Algorithm 1 discussed in Example 4. The red dashed lines give a sparsest cut, while the value h in red gives the Cheeger constant. Edges are added one at a time until a Cheeger constant of 1 is obtained leading from (a) to (d).

$$\partial_1^{(a)} = \begin{pmatrix} 1 & 1 & & & & \\ & 1 & 1 & & & \\ & & & 1 & 1 & \\ & & & & 1 & 1 \end{pmatrix}, \quad \partial_1^{(b)} = \begin{pmatrix} 1 & 1 & & & & \\ & 1 & 1 & & & \\ & & & 1 & 1 & \\ & & & & 1 & 1 \\ & & 1 & 1 & & \end{pmatrix},$$

$$\partial_1^{(c)} = \begin{pmatrix} 1 & 1 & & & & \\ & 1 & 1 & & & \\ & & & 1 & 1 & \\ & & & & 1 & 1 \\ & & 1 & 1 & & \\ 1 & & & & & 1 \end{pmatrix}, \quad \partial_1^{(d)} = \begin{pmatrix} 1 & 1 & & & & \\ & 1 & 1 & & & \\ & & & 1 & 1 & \\ & & & & 1 & 1 \\ & & 1 & 1 & & \\ 1 & & & & & 1 \\ 1 & & & 1 & & \end{pmatrix}.$$

As a consequence of this example, we see that a joint measurement of the X-logical operators across two distance 3 surface codes requires 7 ancillas.

After applying Algorithm 1 to increase the Cheeger constant to 1, one could choose the most sparse option for ∂_0 among all matrices that satisfy $\ker \partial_0 \cong \mathrm{im} \partial_1$; in other words, ∂_0 is chosen to represent a cycle basis for the hypergraph as defined in Definition 4. This choice is analogous to the gauges that are promoted to stabilizers in Ref. [51]. However, when there are redundancies in the Z-stabilizer generators H_Z, this can often be improved. Consider the space of row vectors $V = \{v^T f_0 | v \in \ker H_Z^{\mathrm{T}}\}$ which are cycles that will be represented in $\tilde{H}_Z$ regardless

Algorithm 2. Random search for low weight ∂_0.

Input: ∂_1, H_Z, f_0, number of random samples n
Output: ∂_0
Define V to be any matrix whose rows form a basis of $\{v^T f_0 | v \in \ker H_Z^{\mathrm{T}}\}$.
Put V in reduced row echelon form.
Define W to be any matrix such that $\ker W \cong \mathrm{im}\partial_1$.
Add rows of V to rows of W to zero out the pivot columns of V in W.
Put W in reduced row echelon form with zero-rows removed.
Initialize $\partial_0 \leftarrow W$
for $i \in \{1, \ldots, n\}$ **do**
  Let A be a random, invertible matrix [60].
  Let B be a random (not necessarily invertible) matrix.
  if *the maximum row weight of* $AW + BV$ *is less than that of* ∂_0 **then**
    $\partial_0 \leftarrow AW + BV$
  end
  `// Frequently, not adding rows of V gives lower weight. Check for this:`
  if *the maximum row weight of* AW *is less than that of* ∂_0 **then**
    $\partial_0 \leftarrow AW$
  end
end
return ∂_0

of the choice of ∂_0. With these in mind, choose ∂_0 such that the rows represent a basis for a subspace of all cycles W such that $V \oplus W$ is the entire cycle space of the graph defined by ∂_1. A method for randomly searching such choices of ∂_0 for low weight options is given in Algorithm 2. If this is sparse enough, then the construction is complete.

Unfortunately, there is no guarantee that ∂_0 will be sparse even for a qLDPC input code, in which case more work must be done. In order to improve the sparsity of ∂_0, take a step back and start with the incidence matrix ∂_1^* as defined in Eq. (47) before Algorithm 1 has been applied, take all edges of the hypergraph that are defined on more than two vertices and replace them with any collection of (usual) edges defined only on two vertices each such that all vertices of the original edge are represented. For an edge defined on v vertices, it suffices to choose $v/2$ edges for the replacement. The choice of these $v/2$ edges should be made such that the Cheeger constant is kept as high as possible. Representing all vertices from the original edges is necessary so that the row space of ∂_1^* still includes $H_Z|_Q$ as required by Eq. (36), and f_0 must be modified to satisfy Eq. (36) by replacing each column corresponding to an expanded hyperedge on v vertices with $v/2$ copies of that column. At this point, Algorithm 1 should be applied to obtain a ∂_1^* corresponding to a graph with only weight-2 edges and a Cheeger constant of 1.

Next, apply Algorithm 2 to obtain ∂_0. Now that we have a usual graph rather than a hypergraph, we can understand the cycles defined by the rows of ∂_0 more easily. If there is

Algorithm 3. Main construction for the edge expanded homological measurement.

Input: A code $\mathcal{C} = \mathrm{CSS}(H_X, H_Z)$ and X-logical operator $\bar{X}$
Output: A code $\tilde{\mathcal{C}}$ with at least the distance of $\mathcal{C}$, with $\bar{X}$ in the stabilizer group, and with all other logical operators unharmed.

Define f_1, ∂_1^*, and f_0^* as in Eqs. (35), (47), and (48).
Apply Algorithm 1 to the incidence matrix ∂_1^* to obtain a new incidence matrix ∂_1.
Add zero columns to f_0^* corresponding to the new edges from the previous step obtaining f_0
Apply Algorithm 2 to ∂_1, H_Z, and f_0 to obtain ∂_0
if *the sparsity of* ∂_0 *is deemed unacceptable* **then**
  $\partial_1 \leftarrow \partial_1^*$
  $f_1 \leftarrow f_1^*$
  `// Expand hyperedges to weight-two edges`
  for *each row* e *of* ∂_1 with wt $e > 2$ **do**
    Replace e with $(\mathrm{wt} e)/2$ weight-two rows that sum to e that keep the Cheeger constant as high as possible.
    Replace the column of f_0 that corresponds to e with $(\mathrm{wt}\, e)/2$ copies of that column.
  end
  Apply Algorithm 1 to the incidence matrix ∂_1 which adds new edges to ∂_1.
  Add zero columns to f_0 corresponding to the new edges from the previous step.
  `// find a cycle basis`
  Apply Algorithm 2 to ∂_1, H_Z, and f_0 to obtain ∂_0
  `cellulate large cycles`
  for *each row* c of ∂_0 *with* wt$c$ *higher than desired* **do**
    Add new edges (rows of ∂_1, along with corresponding zero-columns of f_0) within the cycle defined by c to break it into smaller cycles. This results in replacing the high weight row c with multiple lower weight rows corresponding to the new cycles.
  end
end
Define $\tilde{\mathcal{C}}$ to be the mapping cone of f as in Eq. (12)
return $\tilde{\mathcal{C}}$

no cycle basis where all cycles are low weight, we can add edges across vertices inside of large cycles to create multiple smaller cycles, thus reducing the weight of ∂_0, a process called *cellulation*. Here, the construction is complete and we obtain the final f_1, f_0, ∂_1, and ∂_0. The full construction is summarized in Algorithm 3.

Parallel measurement refers to independent measurements of multiple logical operators, as opposed to a single joint measurement. Parallel measurements can be performed via iterative application of Algorithm 3. When a measurement of an X-type operator is performed, the remaining X-logical operators are unchanged, so the next application of Algorithm 3 is straightforward. However, to measure an X-type and Z-type operator in parallel, after constructing the cone code for measurement of the X-type

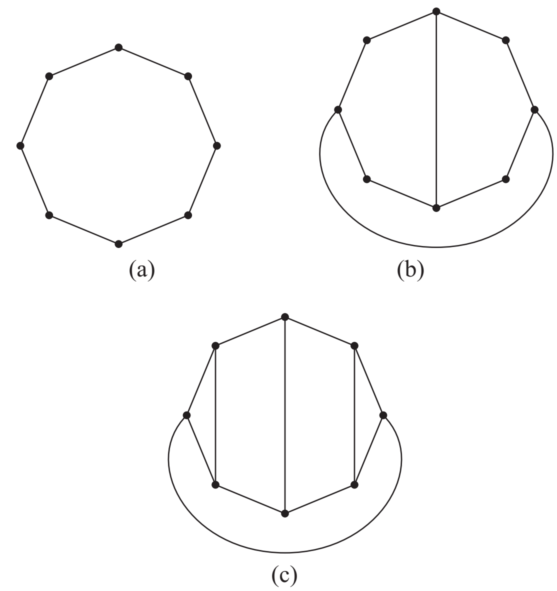

FIG. 2. The graph in (a) has a Cheeger constant of $1/2$. Panel (b) is the result of applying Algorithm 1 to (a) and has a Cheeger constant of 1. In (c), two of the three weight-5 cycles from (b) have been cellulated. See Example 5.

operator, the Z-type operator will have its support extended to some of the ancillas and this must be taken into account.

Remark 4 (Bounds on connectivity in edge expanded homological measurement). The decongestion lemma [61] can be used to build a cycle basis to define ∂_0 such that the connectivity of the cone code in edge expanded homological measurement is increased at most polylogarithmically in the weight of the measured logical operator as compared to the input code. We find that Algorithm 3 tends to result in less connectivity than the decongestion lemma; therefore we suggest the use of Algorithm 3 for practical purposes.

Example 5. Cellulation can be useful for controlling the weights of ∂_0. Suppose we wish to measure a weight-8X-logical operator, and the initial ∂_1^* obtained from Eq. (47) gives a cycle graph with 8 vertices seen in Fig. 2(a) which has Cheeger constant $1/2$. The construction in Ref. [51] would give 24 ancilla qubits and a weight-8Z stabilizer corresponding to the weight of the single cycle in this graph. If the distance of the code is $d = 8$, the construction in Ref. [49] does not create any new stabilizers above weight 4, but requires 120 ancilla qubits. Our edge expanded homological measurement construction results in 10 ancilla qubits without cellulation and 12 with cellulation with maximum stabilizer weights of 5, as seen below.

In our construction, application of Algorithm 1 adds two edges resulting in Fig. 2(b) which increases the Cheeger constant to 1. There is a cycle basis with 3 cycles of weight 5 which is easily seen due to the graph being planar. These three cycles result in 3 stabilizers of weight 5. For concreteness, given a particular ordering of qubits and

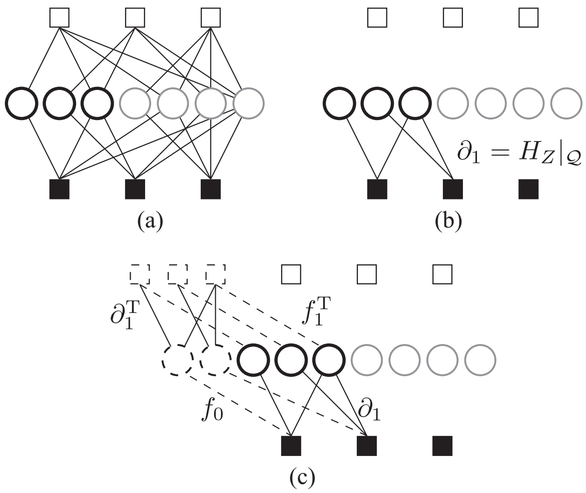

FIG. 3. A Tanner graph depicting measurement in the Steane code of the logical X operator supported on the first three qubits shown with a thick border. The variable nodes with a dashed border represent ancilla. The original code is shown in (a), the restriction of H_Z to the support of the logical operator to obtain ∂_1 is shown in (b), and f_1, f_0, and ∂_1 are shown in (c).

stabilizers of the original code, this would result in the following ∂_1 and ∂_0:

$$\partial_1 = \begin{pmatrix} 1 & 1 & & & & & & \\ & 1 & 1 & & & & & \\ & & 1 & 1 & & & & \\ & & & 1 & 1 & & & \\ & & & & 1 & 1 & & \\ & & & & & 1 & 1 & \\ & & & & & & 1 & 1 \\ 1 & & & & & & & 1 \\ 1 & & & & 1 & & & \\ & & 1 & & & & 1 & \end{pmatrix}, \tag{50}$$

$$\partial_0 = \begin{pmatrix} 1 & 1 & 1 & 1 & & & & & 1 & \\ & & 1 & 1 & 1 & 1 & & & & 1 \\ & & & & 1 & 1 & 1 & 1 & 1 & \end{pmatrix}. \tag{51}$$

We can improve the weights of ∂_0 via cellulation. Since we are measuring an X operator, each new X stabilizer is given by a vertex of the graph, and the weight of the new X stabilizer associated with a vertex is one more than the vertex degree. Therefore, when choosing a cellulation to try to reduce weight-5 operators, we avoid any choices that increase the vertex weights beyond 3. This trade-off between the weights in ∂_1 and ∂_0 is a general feature of cellulation. Given this, the most reasonable cellulation choice is given in Fig. 2(c) where we have reduced this to a single weight-5 cycle. For the same particular ordering of qubits and stabilizers in the above matrices, this gives us the following modified ∂_1 and ∂_0:

$$\partial_1 = \begin{pmatrix} 1 & 1 & & & & & & \\ & 1 & 1 & & & & & \\ & & 1 & 1 & & & & \\ & & & 1 & 1 & & & \\ & & & & 1 & 1 & & \\ & & & & & 1 & 1 & \\ & & & & & & 1 & 1 \\ 1 & & & & & & & 1 \\ 1 & & & & 1 & & & \\ & & 1 & & & & 1 & \\ & 1 & & 1 & & & & \\ & & & & & 1 & & 1 \end{pmatrix}, \tag{52}$$

$$\partial_0 = \begin{pmatrix} 1 & & & 1 & & & & & 1 & & 1 & \\ & 1 & 1 & & & & & & & & 1 & \\ & & 1 & 1 & 1 & 1 & & & & 1 & & \\ & & & & 1 & & & 1 & 1 & & & 1 \\ & & & & & 1 & 1 & & & & & 1 \end{pmatrix}. \tag{53}$$

Example 6. Consider the $[\![7,1,3]\!]$ Steane code given by the following stabilizer matrices:

$$H_X = H_Z = \begin{pmatrix} & & & 1 & 1 & 1 & 1 \\ & 1 & 1 & & & 1 & 1 \\ 1 & & 1 & & 1 & & 1 \end{pmatrix}. \tag{54}$$

Measuring the logical operator $\bar{X} = (1\,1\,1\,0\,0\,0\,0)$ gives the following stabilizer matrices:

$$\widetilde{H}_X = \begin{pmatrix} H_X & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} \end{pmatrix} = \left(\begin{array}{ccccccc|cc} & & & 1 & 1 & 1 & 1 & & \\ & 1 & 1 & & & 1 & 1 & & \\ 1 & & 1 & & 1 & & 1 & & \\ \hline 1 & & & & & & & & 1 \\ & 1 & & & & & & 1 & \\ & & 1 & & & & & 1 & 1 \end{array}\right) \tag{55}$$

$$\widetilde{H}_Z = \begin{pmatrix} H_Z & f_0 \end{pmatrix} = \left(\begin{array}{ccccccc|cc} & & & 1 & 1 & 1 & 1 & & \\ & 1 & 1 & & & 1 & 1 & 1 & \\ 1 & & 1 & & 1 & & 1 & & 1 \end{array}\right). \tag{56}$$

This defines a $[\![9,0]\!]$ code. Note that the last three rows of $\tilde{H}_X$ sum to $(\bar{X}\,0)$. Since k drops to zero, there is no minimum distance to consider. See Fig. 3 for a Tanner graph representation of ∂_1, f_1, and f_0 for this example. Since there are no cycles in the graph given by ∂_1, ∂_0 does not appear in this example.

Example 7. To see when a low Cheeger constant can cause a decrease in distance, consider the $[\![15,7,3]\!]$ quantum Hamming code given by the stabilizer matrices,

$$H_X = H_Z = \begin{pmatrix} &&&&&&&1&1&1&1&1&1&1&1 \\ &&&1&1&1&1&&&&&1&1&1&1 \\ &1&1&&&1&1&&&1&1&&&1&1 \\ 1&&1&&1&&1&&1&&1&&1&&1 \end{pmatrix}, \tag{57}$$

and the logical-X operator $\bar{X} = X_3X_4X_5X_{12}X_{14}$. Without applying Algorithm 1 to improve the Cheeger constant of the graph given by ∂_1, we obtain the $[\![19,6,2]\!]$ code with stabilizer matrices:

$$\tilde{H}_X = \begin{pmatrix} &&&&&&&1&1&1&1&1&1&1&1&&&& \\ &&&1&1&1&1&&&&&1&1&1&1&&&& \\ &1&1&&&1&1&&&1&1&&&1&1&&&& \\ 1&&1&&1&&1&&1&&1&&1&&1&&&& \\ &&1&&&&&&&&&&&&&&&1&1 \\ &&&1&&&&&&&&&&&&&1&& \\ &&&&1&&&&&&&&&&&&1&&1 \\ &&&&&&&&&&&1&&&&1&1&& \\ &&&&&&&&&&&&&1&&1&1&1& \end{pmatrix}, \tag{58}$$

$$\tilde{H}_Z = \begin{pmatrix} &&&&&&&1&1&1&1&1&1&1&1&1&&& \\ &&&1&1&1&1&&&&&1&1&1&1&&1&& \\ &1&1&&&1&1&&&1&1&&&1&1&&&1& \\ 1&&1&&1&&1&&1&&1&&1&&1&&&&1 \end{pmatrix}. \tag{59}$$

The weight-2 logical operators of the new code are given by X_1X_{19}, X_2X_{18}, and X_8X_{16}. The Cheeger constant of the hypergraph given by ∂_1 is 0.5. Only two additional edges (and therefore two additional ancilla), which can be found by applying Algorithm 1, are necessary to maintain the distance.

C. Fault-tolerant protocol for homological measurements

The mathematical framework we have described is a good tool for describing the final state of the system, but does not prescribe a specific protocol. In this section, we describe the steps starting from the input code described by check matrices H_X and H_Z and leading to the final code described by $\tilde{H}_X$ and $\tilde{H}_Z$ given in Eq. (12). It closely follows the steps used to perform lattice surgery introduced in Ref. [44]. This protocol applies to any homological measurement, not just edge expanded homological measurement.

(1) Initialize ancilla. Prepare the ancilla qubits A_0 in the $|0\rangle^{\otimes(\dim A_0)}$ state for an X measurement (or $|+\rangle^{\otimes(\dim A_0)}$ for a Z measurement). At this point, the physical system is stabilized by

$$H'_X = (\,H_X \quad 0\,), \qquad H'_Z = \begin{pmatrix} H_Z & \\ & I \end{pmatrix}. \tag{60}$$

(2) Measure the new X stabilizers. Measure each of the new X stabilizers from the rows $(f_1^{\mathrm{T}}\,\partial_1^{\mathrm{T}})$ of $\tilde{H}_X$. The measurement results are uniformly random and must be recorded. Since

$$\text{row space}\begin{pmatrix} H_Z & \\ & I \end{pmatrix} = \text{row space}\begin{pmatrix} H_Z & f_0 \\ & I \end{pmatrix}, \tag{61}$$

and the rows $(H_Z\,f_0)$ commute with the measurements by construction, only the rows $(0\,I)$ of the latter matrix anticommute and are replaced with $(0\,G)$, where the rows of G generate the subspace that commutes with ∂_1^{T}; i.e., G is a matrix such that $\mathrm{im}\partial_1 = \ker G \subseteq \ker \partial_0$. The result is a physical system that is stabilized by

$$H'_X = \begin{pmatrix} H_X & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} \end{pmatrix}, \qquad H'_Z = \begin{pmatrix} H_Z & f_0 \\ & G \end{pmatrix}, \tag{62}$$

where the Z stabilizers $(0\,\partial_0)$ are included in the row space of $(0\,G)$.

(3) Measure the Z stabilizers. Measure the stabilizers $(H_Z\,f_0)$. If row space$(\partial_0) \subsetneq$ row space(G), treat the stabilizers in row space$(G)\backslash$row space(∂_0) as fixed gauges and do not measure them to avoid obtaining potentially high-weight stabilizer generators.

(4) Measure the stabilizers of $\tilde{C}$ multiple times and perform error correction. While measuring the new stabilizers in $\tilde{H}_X$ gives random results since they anticommute with some existing Z-type stabilizers, their product gives the logical operator being measured. In the ideal case, that would be sufficient to infer the results of the logical measurement with certainty, but in the presence of errors the measurement of the stabilizers can be repeated over time a sufficient amount of time, typically a number of times equal to the distance of the initial code, and the results fed into a decoder in order to infer with high confidence the outcome of the logical measurement. Since in general there are other logical qubits remaining in the code, this is also required to maintain their protection.

(5) Measure out ancillas to return to original code. Measure the ancillas in the Z basis. Use the resulting values to determine the resulting code space. Note that this must be done in a fault-tolerant way, since the measurements can be prone to errors.

D. Jointly measuring a mixture of *X*-, *Y*-, and *Z*-logical operators

To measure a logical operator L that is a mixture of X-, Y-, and Z-logical operators, write the operator as a product of X- and Z-logical operators (including phases if required). Note that one needs to keep track of an overall sign in order to interpret the logical measurement correctly. First, apply Algorithm 1 separately for $\bar{X}$ and $\bar{Z}$ obtaining $f_1^{(X)}$, $f_0^{(X)}$, and $\partial_1^{(X)}$ for $\bar{X}$ and $f_1^{(Z)}$, $f_0^{(Z)}$, and $\partial_1^{(Z)}$ for $\bar{Z}$. Note that the roles of H_X and H_Z are swapped when measuring $\bar{Z}$. We do not find a cycle basis yet at this point. Consider the following matrices which are *not* the final product:

$$\tilde{H}_X^{(\text{first step})} = \begin{pmatrix} H_X & & f_0^{(Z)} \\ f_1^{(X)\mathrm{T}} & \partial_1^{(X)\mathrm{T}} & \end{pmatrix}, \tag{63}$$

$$\tilde{H}_Z^{(\text{first step})} = \begin{pmatrix} H_Z & f_0^{(X)} & \\ f_1^{(Z)\mathrm{T}} & & \partial_1^{(Z)\mathrm{T}} \end{pmatrix}. \tag{64}$$

There are two possibilities after this first step. Either the two matrices do not commute, or if they do, the code they implement would measure $\bar{X}$ and $\bar{Z}$ separately. To fix both problems, we will merge some pairs of stabilizers, i.e., replace stabilizer generators with their product. If the matrices anticommute, then this will be due to stabilizer generators with a single overlapping qubit in the blocks $f_1^{(X)\mathrm{T}}$ and $f_1^{(Z)\mathrm{T}}$, corresponding to the physical Pauli-Y operators that are being measured. Merge each anticommuting pair of stabilizer generators. If the matrices commute, merge any X stabilizer from the rows starting with $f_1^{(X)\mathrm{T}}$ with any Z stabilizer from the rows starting with $f_1^{(Z)\mathrm{T}}$. Any choice will have the desired outcome, so choose the lowest-weight stabilizers. This results in a valid non-CSS stabilizer code given by the following stabilizer matrix in symplectic form where $f_1^{(X\backslash Z)}$, $f_1^{(Z\backslash X)}$, $\partial_1^{(X\backslash Z)}$, and $\partial_1^{(Z\backslash X)}$ are what remains after merging, and $f_1^{(Y|x)}$, $f_1^{(Y|z)}$, $\partial_1^{(Y|x)}$, and $\partial_1^{(Y|z)}$ are created according to the merging:

$$\left(\begin{array}{ccc|ccc} H_X & & f_0^{(Z)} & & & \\ f_1^{(X\backslash Z)^{\mathrm{T}}} & \partial_1^{(X\backslash Z)^{\mathrm{T}}} & & & & \\ f_1^{(Y|x)^{\mathrm{T}}} & \partial_1^{(Y|x)^{\mathrm{T}}} & & f_1^{(Y|z)^{\mathrm{T}}} & & \partial_1^{(Y|z)^{\mathrm{T}}} \\ & & & H_Z & f_0^{(X)} & \\ & & & f_1^{(Z\backslash X)^{\mathrm{T}}} & & \partial_1^{(Z\backslash X)^{\mathrm{T}}} \end{array}\right) \tag{65}$$

This code does include L in its stabilizer group, but it also contains new logical operators that may be low weight, which is the same issue that arises in Sec. III B requiring ∂_0. To fix this, we return to the topic of finding a cyclic basis as in Sec. III B. The new undesired logical operators are given by the cycles of a particular graph. This graph is given by the union of the graphs defined by $\partial_1^{(X)}$ and $\partial_1^{(Z)}$ with vertices between the two identified if their respective stabilizers were merged; i.e., the incidence matrix of the graph is

$$\partial_1 = \begin{pmatrix} \partial_1^{(X\setminus Z)} & \partial_1^{(Y|x)} & \\ & \partial_1^{(Y|z)} & \partial_1^{(Z\setminus X)} \end{pmatrix}. \tag{66}$$

Find a cycle basis for the graph using Algorithm 2 which we notate

$$\partial_0 = \begin{pmatrix} \partial_0^{(X)} & \partial_0^{(Z)} \end{pmatrix}, \tag{67}$$

where edges from the top (bottom) block row in Eq. (66) are represented in $\partial_0^{(X)}$ ($\partial_0^{(Z)}$). If the row weights of ∂_0 are higher than desired, cellulation can be applied where now one may choose for each added edge whether to include it in the graph for X or the graph for Z. The resulting code is given by the following stabilizer matrix in symplectic form:

$$\left(\begin{array}{ccc|ccc} H_X & & f_0^{(Z)} & & & \\ {f_1^{(X\setminus Z)}}^{\mathrm{T}} & {\partial_1^{(X\setminus Z)}}^{\mathrm{T}} & & & & \\ {f_1^{(Y|x)}}^{\mathrm{T}} & {\partial_1^{(Y|x)}}^{\mathrm{T}} & & {f_1^{(Y|z)}}^{\mathrm{T}} & & {\partial_1^{(Y|z)}}^{\mathrm{T}} \\ & & & H_Z & f_0^{(X)} & \\ & & & {f_1^{(Z\setminus X)}}^{\mathrm{T}} & & {\partial_1^{(Z\setminus X)}}^{\mathrm{T}} \\ & & \partial_0^{(Z)} & & \partial_0^{(X)} & \end{array}\right) \tag{68}$$

The dimension of the resulting code is one less than that of the input code by construction. To see that the resulting distance is always preserved, consider an error $\vec{e}$ written in symplectic form and decompose it into $\vec{e} = \vec{e}_X + \vec{e}_Z$ for the X and Z parts of the error. Note that $\mathrm{wt}\vec{e} \geq \min\{\mathrm{wt}\vec{e}_X, \mathrm{wt}\vec{e}_Z\}$ and apply the arguments in the proof of Theorem 1 to $\vec{e}_X$ and $\vec{e}_Z$ independently. The distance will often increase because the X-type measurement frequently increases the Z distance, and vice versa.

IV. EXAMPLES AND NUMERICAL SIMULATIONS

In this section, we first provide the parameters of the codes obtained after the logical measurements have been performed on a few examples. We next benchmark our fault-tolerant logical operator measurement protocol for a measurement-based quantum computer that uses GKP qubits concatenated in various quantum LDPC code.

A. Examples

We consider two classes of quantum LDPC codes: (a) lifted product (LP) codes [23,62] and (b) hypergraph product (HGP) codes [58,63]. We focus on these particular code families as their distances scale with n, they have constant encoding rate [23,58,64], and they perform well with the photonic architecture we consider here, though we emphasize that the procedure presented in Sec. III can be used with any code. To demonstrate homological measurement through our examples, for each code, we consider an X-logical operator for which the weight is equal to the distance of its code, but they have been otherwise chosen randomly.

Lifted product codes. The first class of codes that we consider for our example are the lifted product codes which were first proposed by Panteleev and Kalachev [23]. The lifted product code construction is the lifted version of the hypergraph product code construction.

Given a choice of lift size ℓ, consider the polynomial quotient ring $R_\ell = \mathbb{F}_2[x]/(x^\ell - 1)$. For a polynomial $g(x) = g_0 + g_1 x + \cdots + g_{\ell-1}x^{\ell-1} \in R_\ell$, define $g^{\mathrm{T}}(x) = g_0 + g_{\ell-1}x + \cdots + g_1 x^{\ell-1} \in R_\ell$. The lift of $g(x)$ is an $\ell \times \ell$ circulant matrix $\mathbb{B}[g(x)]$ where the first column is given by the coefficients of $g(x)$ and each subsequent column is obtained by a cyclic shifting down one index; i.e., the ith column is given by the coefficients of $x^{i-1}g(x)$. Similarly, the lift $\mathbb{B}(A)$ of a matrix A over R_ℓ is obtained by replacing each element of A by its lift.

Let A_1 and A_2 be two matrices of size $m_1 \times n_1$ and $m_2 \times n_2$, respectively, with entries in R_ℓ. The quasicyclic lifted product code $\mathrm{LP}(A_1, A_2)$ is the CSS code of length $\ell(n_1 m_2 + n_2 m_1)$ with stabilizer matrices:

$$H_X = \mathbb{B}\left(\begin{bmatrix} A_1 \otimes I & I \otimes A_2 \end{bmatrix}\right), \tag{69}$$

$$H_Z = \mathbb{B}\left(\begin{bmatrix} I \otimes A_2^{\mathrm{T}} & A_1^{\mathrm{T}} \otimes I \end{bmatrix}\right). \tag{70}$$

In these simulations, we choose lifted product codes given by matrices $A_1 = A_2^T = A$ whose entries are monomials in R_ℓ. The matrices are explicitly shown in Table I.

Hypergraph product codes. The second class of codes that we consider for our simulations are the hypergraph product codes, first proposed by Tillich and Zémor [58]. The Tanner graph of a hypergraph product code is constructed by taking the graph product of the Tanner graphs of two classical codes.

TABLE I. Matrices defining the lifted product codes used for simulations.

LP code	ℓ	Matrix over R_ℓ
LP_1	7	$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & x & x^2 & x^5 \\ 1 & x^6 & x^3 & x \end{pmatrix}$
LP_2	9	$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & x & x^6 & x^7 \\ 1 & x^4 & x^5 & x^2 \end{pmatrix}$

Consider classical codes C_1 and C_2, and their respective parity-check matrices H_1 and H_2. Let C_i have parameters $[n_i, k_i, d_i]$. The transposed code of C_i, denoted by C_i^{T}, is given by the parity-check matrix H_i^{T} and has parameters $[m_i, k_i^{\mathrm{T}}, d_i^{\mathrm{T}}]$. When H_i^{T} is a full-rank matrix, C_i^{T} encodes no logical information and has infinite distance, i.e., $k_i^{\mathrm{T}} = 0$ and $d_i^{\mathrm{T}} = \infty$. The hypergraph product code of C_1 and C_2 is denoted $\mathrm{HGP}(C_1, C_2)$ and has stabilizer matrices,

$$H_X = \begin{pmatrix} H_1 \otimes I & I \otimes H_2^{\mathrm{T}} \end{pmatrix}, \tag{71}$$

$$H_Z = \begin{pmatrix} I \otimes H_2 & H_1^{\mathrm{T}} \otimes I \end{pmatrix}, \tag{72}$$

and parameters,

$$[\![n_1 n_2 + m_1 m_2, k_1 k_2 + k_1^T k_2^T, \min(d_1, d_2, d_1^T, d_2^T)]\!]. \tag{73}$$

The specific HGP codes we consider are built from $H = H_1 = H_2$, where H is a parity-check matrix of a classical LDPC code with column weights of four and row weights of three. These classical LDPC codes were obtained using the MacKay-Neal method [63,65]. We consider two hypergraph product codes, denoted HGP_1 and HGP_2, given by the following parity-check matrices:

$$H_{\mathrm{HGP}_1} = \begin{pmatrix}
0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\
0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0
\end{pmatrix}, \tag{74}$$

and

$$
H_{\mathrm{HGP}_2} = \begin{pmatrix}
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\
1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\
0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0
\end{pmatrix}. \tag{75}
$$

The logical operators measured using our new technique are shown in Table II. The code parameters and row and column weights of the original codes are presented in Table III, and the code parameters and weights obtained postmeasurement are presented in Table IV.

TABLE II. Logical operators used for fault-tolerant logical measurement simulations.

Code	Logical operator
LP_1	$X_{30}X_{37}X_{38}X_{39}X_{44}X_{45}X_{47}X_{51}X_{53}X_{54}$
LP_2	$X_{41}X_{42}X_{53}X_{55}X_{56}X_{60}X_{62}X_{63}X_{64}X_{65}X_{68}X_{72}$
HGP_1	$X_{282}X_{286}X_{288}X_{290}X_{292}X_{293}X_{298}X_{300}$
HGP_2	$X_{217}X_{219}X_{220}X_{222}X_{224}X_{225}X_{230}X_{232}X_{235}X_{237}$

B. Numerical simulations

We benchmark our homological measurement protocol for the various codes presented in Sec. IV A via simulation of a photonic architecture based on optical GKP states. The quantum code considered is first foliated (using a number of layers equal to the code distance) to give a cluster state as described in Ref. [40]. The cluster is generated by sending optical GKP qubits through a passive linear optics circuit comprising of beam splitters and phase shifters, and homodyne measurements performed on a subset of the modes. The detailed description of the procedure used can be found in Refs. [33,41]. We provide a brief discussion of the photonic quantum computing architecture used for the simulations below.

(1) Preparing GKP Bell pair. A GKP Bell pair is constructed using two GKP sensor states, a 50∶50

TABLE III. Properties of the simulated codes.

Code	(q_X, w_X, q_Z, w_Z)	$(q,\ w)$	$(q_{X_{\mathrm{av}}}, w_{X_{\mathrm{av}}}, q_{Z_{\mathrm{av}}}, w_{Z_{\mathrm{av}}})$	$(q_{\mathrm{av}}, w_{\mathrm{av}})$
LP_1, $[\![175, 19, 10]\!]$	(4, 7, 4, 7)	(8, 7)	(3.36, 7, 3.36, 7)	(6.72, 7)
LP_2, $[\![225, 21, 12]\!]$	(4, 7, 4, 7)	(8, 7)	(3.36, 7, 3.36, 7)	(6.72, 7)
HGP_1, $[\![625, 25, 8]\!]$	(4, 7, 4, 7)	(8, 7)	(3.36, 7, 3.36, 7)	(6.72, 7)
HGP_2, $[\![900, 36, 10]\!]$	(4, 7, 4, 7)	(8, 7)	(3.36, 7, 3.36, 7)	(6.72, 7)

TABLE IV. Properties of the codes obtained post fault-tolerant logical measurements.

Original code	Measured code	$(\tilde{q}_X, \tilde{w}_X, \tilde{q}_Z, \tilde{w}_Z)$	(q, w)	$(\tilde{q}_{X_{\rm av}}, \tilde{w}_{X_{\rm av}}, \tilde{q}_{Z_{\rm av}}, \tilde{w}_{Z_{\rm av}})$	$(\tilde{q}_{\rm av}, \tilde{w}_{\rm av})$
LP_1, $[[175, 19, 10]]$	$[[191, 18, 10]]$	(4, 7, 6, 9)	(10, 9)	(3.31, 6.72, 3.39, 7.19)	(6.70, 6.95)
LP_2, $[[225, 21, 12]]$	$[[245, 20, 12]]$	(4, 7, 7, 12)	(9, 12)	(3.30, 6.73, 3.47, 7.34)	(6.77, 7.03)
HGP_1, $[[625, 25, 8]]$	$[[638, 24, 8]]$	(4, 7, 5, 8)	(9, 8)	(3.35, 6.94, 3.37, 7.03)	(6.72, 6.98)
HGP_2, $[[900, 36, 10]]$	$[[917, 35, 10]]$	(4, 7, 7, 11)	(9, 11)	(3.35, 6.94, 3.39, 7.06)	(6.73, 7.00)

beam splitter, and a $\pi/2$ phase shifter. We model the generation of the GKP Bell pair by considering two modes in the ideal GKP sensor state, applying a Gaussian blurring channel of variance σ^2 that replaces each delta function of the GKP state by a Gaussian distribution of variance σ^2, and applying the optical circuit comprising of a 50:50 beam splitter and a $\pi/2$ phase shifter. Throughout this paper, we express the variance of the Gaussian in terms of decibels, $[\sigma^2/(\sigma^2_{\rm vac})][\mathrm{dB}] = -10\log_{10}(\sigma^2/\sigma^2_{\rm vac})$, where $\sigma^2_{\rm vac}$ is the variance of the vacuum, which is $1/2$ in units where $\hbar = 1$. This model is equivalent to uniform photon loss experienced throughout the cluster state generation and measurement process.

(2) Preparing the optical resource state. The optical resource state is obtained by considering the cluster state based on the foliated quantum code, replacing each edge of the cluster state by a GKP Bell pair, and applying a beam splitter circuit on all the modes that correspond to the same node in the cluster state to obtain the resource state. The set of modes in various GKP Bell pairs corresponding to the same node in the cluster state are called a macronode. The beam splitter circuit corresponds to performing a continuous-variable Greenberger-Horne-Zeilinger measurement.

(3) Performing measurement-based quantum computation. The GKP cluster state is obtained from the optical resource state by performing a homodyne measurement on all but one mode in each macronode in the q quadrature. Homodyne measurements in the p quadrature are performed on the remaining mode in each macronode as the quantum computer is considered to be operating in quantum error correction mode. The quantum error correction mode involves performing X basis measurements on all qubits in the cluster state. To perform a Z basis measurement when the quantum computer is in another mode, the remaining mode in the macronode should be measured in the q quadrature. We note that the qubit Pauli error rate of a qubit increases monotonically with variance σ^2 (decreases monotonically with $\sigma^2/\sigma^2_{\rm vac}[\mathrm{dB}]$) and the degree of the node in the cluster state.

(4) Applying feedforward corrections. The homodyne measurements are processed to obtain the GKP qubit measurement value in the GKP cluster state. This involves first processing measurement outcomes in the same mode in the macronode. Each processed measurement outcome is decoded using a soft-in soft-out GKP decoder, which is the binning function described in Ref. [33] to obtain a bit value and a

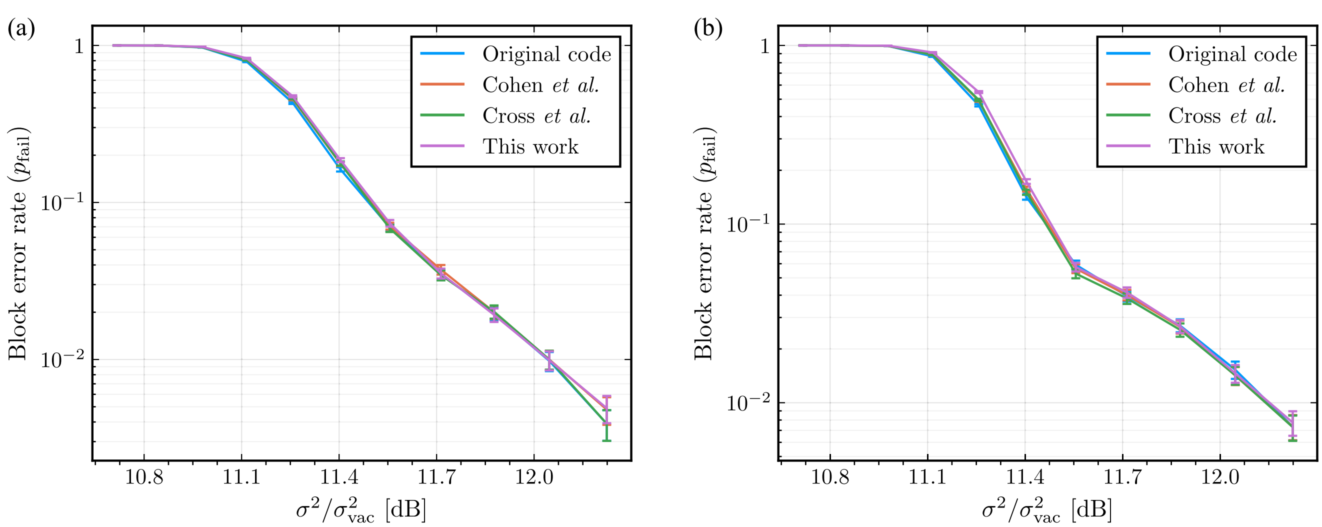

FIG. 4. Block error rates of lifted product codes before and after a logical-X measurement. (a) [[175, 19, 10]] lifted product code, LP_1. (b) [[225, 21, 12]] lifted product code, LP_2.

TABLE V. Comparison of the number of ancillas required for different measurement techniques. The logical operators that were measured are given in Table II.

Code	n_{anc}, Cohen *et al.* [49]	n_{anc}, Cross *et al.* [51]	n_{anc}, this work
LP_1, $[[175, 19, 10]]$	230	38	16
LP_2, $[[225, 21, 12]]$	348	48	20
HGP_1, $[[625, 25, 8]]$	144	30	13
HGP_2, $[[900, 36, 10]]$	240	40	17

probability of error. Feedforward corrective displacements are needed to obtain the bit value and probability of error corresponding to the GKP qubits in the GKP cluster state. These are obtained by combining the bit values and probability of error of a particular mode of a macronode with a few modes of the macronode corresponding to its neighbors in the cluster state to obtain the qubit value and the probability of qubit error.

(5) Syndrome computation and qubit decoding. The syndrome is computed from these bit values and the qubit weight w_q is computed from the probability of qubit error p_q as $w_q = \log[(1-p_q)/p_q]$. The syndrome and the qubit weights are used to perform soft-in hard-out qubit decoding using the foliated quantum code. We use the belief propagation (BP) ordered statistics decoder (OSD) to perform qubit decoding.

We use the min-sum algorithm for BP decoding with N iterations and a flooding schedule, where N is the number of qubits in the foliated cluster state. When the BP decoding does not converge, we use an OSD with combination sweep strategy with search depth parameter $\lambda = 60$. As our homological measurement technique can be used across the class of quantum LDPC codes, we use the BP OSD decoder, which can be used with all quantum LDPC codes.

We use logical block error rate to quantify the performance of our architecture encoded with the example codes before and after the homological measurements. We perform Monte Carlo simulations to obtain estimates of the logical error rate.

For the logical block error rate computation, we consider a logical error to have occurred in a trial when any logical qubit has an error after decoding. Denoting the number of times at least a logical error occurs by n_{fail} and n_{tot} the total number of trials, using the Agresti-Coull interval [66,67], the logical block error rate is estimated by

$$p_{\mathrm{fail}} = \frac{n_{\mathrm{fail}} + \kappa^2/2}{n_{\mathrm{tot}} + \kappa^2}, \tag{76}$$

and the uncertainties are estimated by

$$\kappa\sqrt{\frac{p_{\mathrm{fail}}(1-p_{\mathrm{fail}})}{n_{\mathrm{tot}} + \kappa^2}}, \tag{77}$$

with p_{fail} denoting the estimated probability of sustaining a logical block error, and where κ, the desired quantile of a

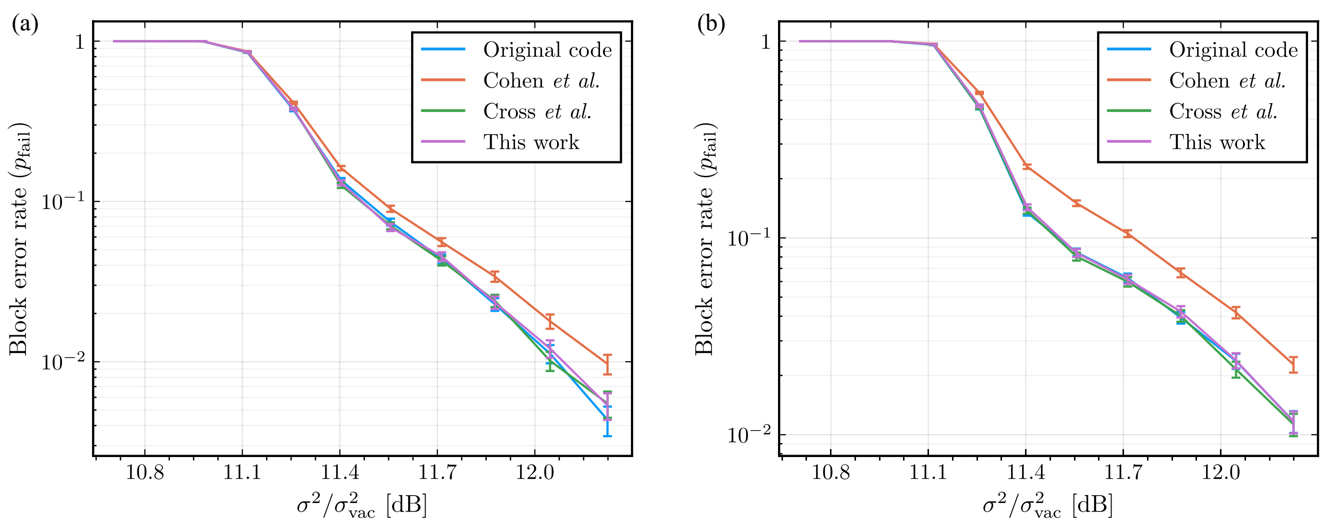

FIG. 5. Performance plot analyzing the block error rates of hypergraph product codes before and after a logical-X measurement. (a) [[625, 25, 8]] hypergraph product code, HGP_1. (b) [[900, 36, 10]] hypergraph product code, HGP_2.

standard normal distribution, is set to $\kappa = 1.96$ corresponding to a 95% confidence interval.

It is worth emphasizing that the Agresti-Coull interval is used rather than the traditional Wald interval because the Wald interval is simply a bad choice. For a large number of trials, such as the present case, the Agresti-Coull interval is highly performant, while in other situations, both the Jeffreys and Wilson intervals should be considered [67].

Figure 4 shows the logical block error rate for the lifted product codes LP_1 and LP_2. Our protocol performs similarly to those presented in Refs. [49,51] while using a smaller number of additional ancilla qubits as shown in Table V. Note that the block error rates are also similar to that of the original codes.

Figure 5 shows the logical block error rate of the hypergraph product codes HGP_1 and HGP_2. For these codes, our scheme performs equivalently to the scheme presented in Ref. [51], while outperforming generalized lattice surgery [49]. This may be due to the fact that the ancilla code used by generalized lattice surgery repeats a potentially bad structure d times that depends on the specific logical representative being measured, in combination with the fact that the ancilla code in this scheme is significantly larger than for the other protocols considered.

V. DISCUSSION

We introduced the notion of homological measurements based on the chain complex description of CSS codes. Several protocols such as lattice surgery, generalized lattice surgery, and some optimized variants thereof fall within the framework of homological measurements. The homological measurement framework provides a natural mathematical language that allowed us to devise a new protocol, the edge expanded homological measurement, which provides fault-tolerant measurement of logical Pauli operators with fewer ancilla qubits than in previously known methods such as generalized lattice surgery [49], its recent refinement [51], and homomorphic measurements [48]. Note that our scheme is general in the sense that it can be used for any CSS code. As it does not require any specific structure, it is likely that our method can be optimized further for combinations of specific codes and logical operators to be measured, similar to the treatment that was given to the gross code in Ref. [51].

The recent work [68] uses many related ideas but they take a larger number of ancilla qubits in general in exchange for a provably qLDPC construction. We quickly examine the case where their construction (for a CSS code) can contain all cellulations within a single layer of their ancilla system for comparison. Let ∂_1' be the incidence matrix of the graph obtained in Algorithm 3 before cellulation is applied, and ∂_1 be the incidence matrix including the cellulation. Let ∂_0, f_1, and f_0 be obtained as in Algorithm 3. While our work results in a merged code with stabilizer matrices,

$$\tilde{H}_X = \begin{pmatrix} H_X & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} \end{pmatrix}, \qquad \tilde{H}_Z = \begin{pmatrix} H_Z & f_0 \\ & \partial_0 \end{pmatrix}, \tag{78}$$

this related work [68] results in

$$\tilde{H}_X = \begin{pmatrix} H_X & & & \\ f_1^{\mathrm{T}} & {\partial_1'}^{\mathrm{T}} & & I \\ & & \partial_1^{\mathrm{T}} & I \end{pmatrix}, \tag{79}$$

$$\tilde{H}_Z = \begin{pmatrix} H_Z & f_0 & & \\ & I & \begin{pmatrix} I & 0 \end{pmatrix} & \partial_1' \\ & & \partial_0 & \end{pmatrix}, \tag{80}$$

where the zero columns of $(I0)$ correspond to the edges added for cellulation. We note that viewing this within our framework, a minor modification of Theorem 2 shows that the multigraph with all edges from both ∂_1 and ∂_1' needs to have a Cheeger constant of 1 to maintain distance in the merged code. In this particular case, it implies that a Cheeger constant of only 0.5 is necessary for the graph defined by ∂_1', resulting in a useful optimization. When their construction requires more layers, the required Cheeger constant reduces further. Also note that for CSS codes this technique fits nicely within the homological measurement framework.

We performed numerical simulations of a photonic architecture based on the use of GKP qubits which demonstrated the practicality of the protocol. Since increasing the weight and connectivity of stabilizer measurements strongly negatively affects the effective qubit noise in the architecture on which our simulations are based [31,33,41], and the simulation results show that our protocol is very robust to noise, this demonstrates that the slight increase in connectivity of a limited number of stabilizer generators is not impactful in terms of logical performance. The strong resilience to noise on ancilla qubits used to perform lattice surgery in the case surface code has been previously observed [69], and our results indicate that this may also be the case for more general classes of homological measurements. We have not analyzed the logical performance of edge expanded homological measurements beyond the results presented in Sec. IV, and in particular, we have made no attempt to optimize with regards of logical operator representative or decoding strategies to take into account the specific structures introduced by our protocol. A natural open question is how to optimize for specific logical operator structures leading to better logical performance. We leave such analysis and optimization for future work.

We have focused on designing protocols minimizing the number of ancilla qubits required, but we have not attempted to reduce the time overhead. Our protocol requires measuring the new stabilizers roughly d times to be able to infer reliably the outcome of the logical measurement performed. However, if the initial codes allow for single-shot quantum error correction [70–72], one can wonder whether a scheme could be devised such that only a constant number of measurement rounds are needed. Since metastabilizer CSS codes can readily be described by longer chain complexes, we conjecture that the homological measurement framework will allow us to find protocols reducing the number of measurement rounds required.

Note that the protocol we propose does not guarantee that an input qLDPC code will remain so, similar to the situation in Ref. [51] where no general bound can be given on the weight of the gauges that are promoted to stabilizers. Our focus was on practical constructions for near term devices, and the connectivity obtained for all specific constructions considered during the development of this work were low in practice. Using the decongestion lemma [61], we know that the asymptotic bound on connectivity increases polylogarithmically with distance in the worst case, though this worst case was never observed. While we speculate that a modification of Algorithm 1 can give a provably qLDPC output code when the input is also qLDPC with only minimal extra structural requirements, we leave such exploration for future work. Alternatively, one could apply ideas of quantum weight reduction [31,53,54] on the obtained code to reduce the connectivity to constant values. While such techniques typically incur somewhat large overheads, the number of stabilizers requiring weight reduction might be small enough so that it is still favorable compared to other measurement schemes.

As we presented in the main text, it is straightforward to measure different commuting logical Pauli operators in parallel, though measuring more operators will increase the connectivity of the resulting code in general, which in practice will limit the amount of achievable parallelization. This depends highly on the specific codes as well as logical operator to be measured and must be considered on a case-by-case basis.

Note added. Recently, we became aware of work by Williamson and Yoder [68] on a similar construction.

ACKNOWLEDGMENTS

We thank Eric Sabo and Michael Vasmer for valuable discussions, as well as Rafael Alexander for carefully reviewing the manuscript. We thank Alexander Cowtan for noticing an error in a previous version of the manuscript and for his careful review. We also express our gratitude toward Professor John Sipe for facilitating the access to numerical resources. This research was enabled in part by support provided by Scinet and the Digital Research Alliance of Canada. Computations were performed on the Niagara [73] supercomputer at the SciNet HPC Consortium, as well as on Graham system. SciNet is funded by Innovation, Science and Economic Development Canada; the Digital Research Alliance of Canada; the Ontario Research Fund: Research Excellence; and the University of Toronto.

APPENDIX: MAPPING CYLINDER

In addition to the mapping cone which is used in the main text, we also considered the mapping cylinder. This did not result in a useful measurement scheme. We present the construction here, along with the way in which it fails for implementing logical measurements, in the hope that it might be useful in future work.

The mapping cylinder of a chain map $f\colon \mathcal{A} \to \mathcal{C}$ is the chain complex with spaces $\mathrm{cyl}(f)_i = C_i \oplus A_{i-1} \oplus A_i$ and maps $\partial_i^{\mathrm{cyl}(f)}\colon \mathrm{cyl}(f)_i \to \mathrm{cyl}(f)_{i-1}$ defined by

$$\begin{tikzcd}
A_i \arrow[r, "\partial_i^{\mathcal{A}}"] & A_{i-1} \\
\oplus & \oplus \\
A_{i-1} \arrow[r, "\partial_{i-1}^{\mathcal{A}}"] \arrow[uur, "I"] \arrow[ddr, "f_{i-1}"] & A_{i-2} \\
\oplus & \oplus \\
C_i \arrow[r, "\partial_i^{\mathcal{C}}"'] & C_{i-1}
\end{tikzcd} \tag{A1}$$

$$\mathrm{cyl}(f)_i \xrightarrow{\partial_i^{\mathrm{cyl}(f)}} \mathrm{cyl}(f)_{i-1}$$

$$\partial_i^{\mathrm{cyl}(f)} = \begin{array}{c} \\ C_{i-1} \\ A_{i-2} \\ A_{i-1} \end{array} \begin{array}{c} \begin{array}{ccc} C_i & A_{i-1} & A_i \end{array} \\ \begin{pmatrix} \partial_i^{\mathcal{C}} & -f_{i-1} & 0 \\ 0 & -\partial_{i-1}^{\mathcal{A}} & 0 \\ 0 & I & \partial_i^{\mathcal{A}} \end{pmatrix} \end{array}.$$

The minus signs in $\partial_i^{\mathrm{cyl}(f)}$ can be dropped when working in $\mathbb{F}_2$.

Note that we can interpret the mapping cylinder as the mapping cone of a different chain map. Define the chain complex $\mathcal{B}$ with spaces $B_i = A_i \oplus A_{i+1}$ and maps

$$\partial_i^{\mathcal{B}} = \begin{pmatrix} \partial_i^{\mathcal{A}} & \\ -I & -\partial_{i+1}^{\mathcal{A}} \end{pmatrix} \tag{A2}$$

and the chain map $g\colon \mathcal{B} \to \mathcal{C}$ given by

$$g_i = \begin{pmatrix} f_i & 0 \end{pmatrix}. \tag{A3}$$

It can be seen that $\mathrm{cyl}(f) = \mathrm{cone}(g)$.

Suppose that we have a CSS code $\mathrm{CSS}(H_X, H_Z)$ viewed as a chain,

$$\mathcal{C}\colon\ C_2 \xrightarrow{H_X^{\mathrm{T}}} C_1 \xrightarrow{H_Z} C_0, \tag{A4}$$

an ancilla chain,

$$\mathcal{A}\colon\ A_2 \xrightarrow{\partial_2} A_1 \xrightarrow{\partial_1} A_0 \xrightarrow{\partial_0} A_{-1}, \tag{A5}$$

and a chain map $f: \mathcal{A} \to \mathcal{C}$. Define the cylinder code by the diagram

$$\begin{tikzcd}
A_2 \arrow[r, "\partial_2"] & A_1 \arrow[r, "\partial_1"] & A_0 \\
\oplus & \oplus & \oplus \\
A_1 \arrow[r, "\partial_1"] \arrow[uur, "I"] \arrow[ddr, "f_1"] & A_0 \arrow[r, "\partial_0"] \arrow[uur, "I"] \arrow[ddr, "f_0"] & A_{-1} \\
\oplus & \oplus & \oplus \\
C_2 \arrow[r, "H_X^{\mathrm{T}}"'] & C_1 \arrow[r, "H_Z"'] & C_0
\end{tikzcd} \tag{A6}$$

which has stabilizer matrices

$$\tilde{H}_X = \begin{pmatrix} H_X & & \\ f_1^{\mathrm{T}} & \partial_1^{\mathrm{T}} & I \\ & & \partial_2^{\mathrm{T}} \end{pmatrix}, \qquad \tilde{H}_Z = \begin{pmatrix} H_Z & f_0 & \\ & \partial_0 & \\ & I & \partial_1 \end{pmatrix}. \tag{A7}$$

Note that since $\partial_0 \partial_1 = 0$, all rows resulting from ∂_0 in $\tilde{H}_Z$ are redundant, so we can instead choose

$$\tilde{H}_Z = \begin{pmatrix} H_Z & f_0 & \\ & I & \partial_1 \end{pmatrix}. \tag{A8}$$

Since this is a special case of the cone, we know that Z distance is maintained by Theorem 1. If f_1, ∂_1, and f_0 are given as in Eqs. (35), (47), and (48) (without any improvements to the Cheeger constant) and $\dim \ker \partial_1 = 1$, it can be seen that the X distance is maintained without any other restrictions, and in particular, this holds regardless of the Cheeger constant of the graph defined by treating ∂_1 as an incidence matrix. Also, since $\partial_1 \partial_2 = 0$, there are only two choices for ∂_2 in this case. It is either 0, or the all-ones vector. In the latter case, this has the same weight as the logical to measure and so it is not useful, and in the former case, the logical we wished to measure is not in the row space of $\tilde{H}_X$.

Note that by choosing $\partial_2 = 0$ and promoting the gauge this creates to a Z stabilizer, the $L = 2$ construction from Ref. [51] is obtained (though because the X distance is maintained regardless, there is no need to promote the gauge to a stabilizer). For larger even values of L, there continues to be no way to get the logical into the row space of $\tilde{H}_X$. Note also that for even values of L, there is no reason to go beyond $L = 2$ because the purpose of increasing L is to maintain distance. We do not believe that this construction allows for a stand-alone measurement protocol. While the mapping cylinder is not useful for directly measuring logical operators, the structure is related to two joint measurement schemes [59,74].

It is our hope, inspired by the utility of the mapping cone in the main text, that formalizing the idea of the mapping cylinder may also be of use in future applications.

[1] P. W. Shor, *Scheme for reducing decoherence in quantum computer memory*, Phys. Rev. A **52**, R2493 (1995).

[2] D. Gottesman, *Stabilizer codes and quantum error correction*, Ph.D. thesis, California Institute of Technology, 1997.

[3] A. Kitaev, *Fault-tolerant quantum computation by anyons*, Ann. Phys. (Amsterdam) **303**, 2 (2003).

[4] H. Bombin and M. A. Martin-Delgado, *Topological quantum distillation*, Phys. Rev. Lett. **97**, 180501 (2006).

[5] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, *Topological quantum memory*, J. Math. Phys. (N.Y.) **43**, 4452 (2002).

[6] H. Bombin, R. S. Andrist, M. Ohzeki, H. G. Katzgraber, and M. A. Martin-Delgado, *Strong resilience of topological codes to depolarization*, Phys. Rev. X **2**, 021004 (2012).

[7] S. Bravyi, D. Poulin, and B. Terhal, *Tradeoffs for reliable quantum information storage in 2D systems*, Phys. Rev. Lett. **104**, 050503 (2010).

[8] C. Gidney and M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum **5**, 433 (2021).

[9] I. H. Kim, Y.-H. Liu, S. Pallister, W. Pol, S. Roberts, and E. Lee, *Fault-tolerant resource estimate for quantum chemical simulations: Case study on Li-ion battery electrolyte molecules*, Phys. Rev. Res. **4**, 023019 (2022).

[10] A. A. Kovalev and L. P. Pryadko, *Improved quantum hypergraph-product LDPC codes*, in *Proceedings of the 2012 IEEE International Symposium on Information Theory Proceedings, Cambridge, MA* (2012), pp. 348–352, 10.1109/ISIT.2012.6284206.

[11] J.-P. Tillich and G. Zémor, *Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength*, IEEE Trans. Inf. Theory **60**, 1193 (2014).

[12] D. Gottesman, *Fault-tolerant quantum computation with constant overhead*, Quantum Inf. Comput. **14**, 1338 (2014).

[13] S. Bravyi and M. B. Hastings, *Homological product codes*, in *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing (STOC'14)* (Association for Computing Machinery, New York, 2014), pp. 273–282.

[14] A. Leverrier, J.-P. Tillich, and G. Zémor, *Quantum expander codes*, in *Proceedings of the 2015 IEEE 56th Annual Symposium on Foundations of Computer Science, Berkeley, CA* (2015), pp. 810–824, 10.1109/FOCS.2015.55.

[15] N. P. Breuckmann and B. M. Terhal, *Constructions and noise threshold of hyperbolic surface codes*, IEEE Trans. Inf. Theory **62**, 3731 (2016).

[16] N. P. Breuckmann, C. Vuillot, E. Campbell, A. Krishna, and B. M. Terhal, *Hyperbolic and semi-hyperbolic surface codes for quantum storage*, Quantum Sci. Technol. **2**, 035007 (2017).

[17] O. Fawzi, A. Grospellier, and A. Leverrier, *Constant overhead quantum fault-tolerance with quantum expander codes*, in *Proceedings of the 2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS), Paris, France* (2018), pp. 743–754, 10.1109/FOCS.2018.00076.

[18] W. S. Soares and E. B. Da Silva, *Hyperbolic quantum color codes*, Quantum Inf. Comput. **18**, 306 (2018).

[19] M. B. Hastings, J. Haah, and R. O'Donnell, *Fiber bundle codes: Breaking the* $n1/2$ polylog(n) *barrier for quantum LDPC codes*, in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing* (STOC 2021)

(Association for Computing Machinery, New York, 2021), pp. 1276–1288.

[20] S. Evra, T. Kaufman, and G. Zémor, *Decodable quantum LDPC codes beyond the* $\sqrt{n}$ *distance barrier using high dimensional expanders*, SIAM J. Comput., **53**, 276 (2022).

[21] P. Panteleev and G. Kalachev, *Degenerate quantum LDPC codes with good finite length performance*, Quantum **5**, 585 (2021).

[22] N. P. Breuckmann and J. N. Eberhardt, *Balanced product quantum codes*, IEEE Trans. Inf. Theory **67**, 6653 (2021).

[23] P. Panteleev and G. Kalachev, *Quantum LDPC codes with almost linear minimum distance*, IEEE Trans. Inf. Theory **68**, 213 (2022).

[24] C. Vuillot and N. P. Breuckmann, *Quantum pin codes*, IEEE Trans. Inf. Theory **68**, 5955 (2022).

[25] I. Dinur, M.-H. Hsieh, T.-C. Lin, and T. Vidick, *Good quantum LDPC codes with linear time decoders*, in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing* (Association for Computing Machinery, New York, 2023), pp. 905–918, 10.1145/3564246.3585101.

[26] A. Strikis and L. Berent, *Quantum low-density parity-check codes for modular architectures*, PRX Quantum **4**, 020321 (2023).

[27] J. Roffe, L. Z. Cohen, A. O. Quintavalle, D. Chandra, and E. T. Campbell, *Bias-tailored quantum LDPC codes*, Quantum **7**, 1005 (2023).

[28] S. Yang and R. Calderbank, *Spatially-coupled QLDPC codes*, arXiv:2305.00137.

[29] Y. Feng, C. Tang, and C. Bai, *New quantum LDPC codes based on Euclidean geometry*, Laser Phys. **34**, 065201 (2024).

[30] S. Miao, J. Mandelbaum, H. Jäkel, and L. Schmalen, *A joint code and belief propagation decoder design for quantum LDPC codes*, arXiv:2401.06874.

[31] E. Sabo, L. G. Gunderman, B. Ide, M. Vasmer, and G. Dauphinais, *Weight-reduced stabilizer codes with lower overhead*, PRX Quantum **5**, 040302 (2024).

[32] J. E. Bourassa, R. N. Alexander, M. Vasmer, A. Patil, I. Tzitrin, T. Matsuura, D. Su, B. Q. Baragiola, S. Guha, G. Dauphinais, K. K. Sabapathy, N. C. Menicucci, and I. Dhand, *Blueprint for a scalable photonic fault-tolerant quantum Computer*, Quantum **5**, 392 (2021).

[33] I. Tzitrin, T. Matsuura, R. N. Alexander, G. Dauphinais, J. E. Bourassa, K. K. Sabapathy, N. C. Menicucci, and I. Dhand, *Fault-tolerant quantum computation with static linear optics*, PRX Quantum **2**, 040353 (2021).

[34] S. Bartolucci, P. Birchall, H. Bombin, H. Cable, C. Dawson, M. Gimeno-Segovia, E. Johnston, K. Kieling, N. Nickerson, M. Pant *et al.*, *Fusion-based quantum computation*, Nat. Commun. **14**, 912 (2023).

[35] B. Pankovich, A. Kan, K. H. Wan, M. Ostmann, A. Neville, S. Omkar, A. Sohbi, and K. Brádler, *High-photon-loss threshold quantum computing using GHZ-state measurements*, Phys. Rev. Lett. **133**, 050604 (2024).

[36] R. Raussendorf, S. Bravyi, and J. Harrington, *Long-range quantum entanglement in noisy cluster states*, Phys. Rev. A **71**, 062313 (2005).

[37] R. Raussendorf, J. Harrington, and K. Goyal, *A fault-tolerant one-way quantum computer*, Ann. Phys. (Amsterdam) **321**, 2242 (2006).

[38] R. Raussendorf and J. Harrington, *Fault-tolerant quantum computation with high threshold in two dimensions*, Phys. Rev. Lett. **98**, 190504 (2007).

[39] R. Raussendorf, J. Harrington, and K. Goyal, *Topological fault-tolerance in cluster state quantum computation*, New J. Phys. **9**, 199 (2007).

[40] A. Bolt, G. Duclos-Cianci, D. Poulin, and T. M. Stace, *Foliated quantum error-correcting codes*, Phys. Rev. Lett. **117**, 070501 (2016).

[41] B. W. Walshe, B. Q. Baragiola, H. Ferretti, J. Gefaell, M. Vasmer, R. Weil, T. Matsuura, T. Jaeken, G. Pantaleoni, Z. Han, N. C. Menicucci, I. Tzitrin, and R. N. Alexander, *Linear-optical quantum computation with arbitrary error-correcting codes*, arXiv:2408.04126.

[42] D. Litinski, *A game of surface codes: Large-scale quantum computing with lattice surgery*, Quantum **3**, 128 (2019).

[43] S. Bravyi and A. Kitaev, *Universal quantum computation with ideal Clifford gates and noisy ancillas*, Phys. Rev. A **71**, 022316 (2005).

[44] C. Horsman, A. G. Fowler, S. Devitt, and R. V. Meter, *Surface code quantum computing by lattice surgery*, New J. Phys. **14**, 123011 (2012).

[45] A. J. Landahl and C. Ryan-Anderson, *Quantum computing by color-code lattice surgery*, arXiv:1407.5103.

[46] D. Litinski and F. v. Oppen, *Lattice surgery with a twist: Simplifying Clifford gates of surface codes*, Quantum **2**, 62 (2018).

[47] H. Poulsen Nautrup, N. Friis, and H. J. Briegel, *Fault-tolerant interface between quantum memories and quantum processors*, Nat. Commun. **8**, 1321 (2017).

[48] S. Huang, T. Jochym-O'Connor, and T. J. Yoder, *Homomorphic logical measurements*, PRX Quantum **4**, 030301 (2023).

[49] L. Z. Cohen, I. H. Kim, S. D. Bartlett, and B. J. Brown, *Low-overhead fault-tolerant quantum computing using long-range connectivity*, Sci. Adv. **8**, eabn1717 (2022).

[50] A. Cowtan and S. Burton, *CSS code surgery as a universal construction*, Quantum **8**, 1344 (2024).

[51] A. Cross, Z. He, P. Rall, and T. Yoder, *Linear-size ancilla systems for logical measurements in QLDPC codes*, arXiv:2407.18393.

[52] C. A. Weibel, *An Introduction to Homological Algebra*, Cambridge Studies in Advanced Mathematics No. 38 (Cambridge University Press, Cambridge, England, 1994).

[53] M. B. Hastings, *Weight reduction for quantum codes*, arXiv:1611.03790.

[54] M. B. Hastings, *On quantum weight reduction*, arXiv:2102.10030.

[55] A. Wills, T.-C. Lin, and M.-H. Hsieh, *Tradeoff constructions for quantum locally testable codes*, arXiv:2309.05541.

[56] D. Gottesman, A. Kitaev, and J. Preskill, *Encoding a qubit in an oscillator*, Phys. Rev. A **64**, 012310 (2001).

[57] J. Rotman, *An Introduction to Homological Algebra*, Vol. 2 (Springer, New York, 2009).

[58] J.-P. Tillich and G. Zémor, *Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength*, IEEE Trans. Inf. Theory **60**, 1193 (2014).

[59] G. Zhang and Y. Li, *Time-efficient logical operations on quantum LDPC codes*, arXiv:2408.01339.

[60] D. Randall, *Efficient generation of random nonsingular matrices*, Random Struct. Algorithms **4**, 111 (1993).

[61] M. Freedman and M. Hastings, *Building manifolds from quantum codes*, Geom. Funct. Anal. **31**, 855 (2021).

[62] N. Raveendran, N. Rengaswamy, F. Rozpedek, A. Raina, L. Jiang, and B. Vasić, *Finite rate QLDPC-GKP coding scheme that surpasses the CSS Hamming bound*, Quantum **6**, 767 (2022).

[63] J. Roffe, D. R. White, S. Burton, and E. Campbell, *Decoding across the quantum low-density parity-check code landscape*, Phys. Rev. Res. **2**, 043423 (2020).

[64] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, *High-threshold and low-overhead fault-tolerant quantum memory*, Nature (London) **627**, 778 (2024).

[65] D. MacKay and R. Neal, *Near Shannon limit performance of low density parity check codes*, Electron. Lett. **32**, 1645 (1996).

[66] A. Agresti and B. A. Coull, *Approximate is better than "exact" for interval estimation of binomial proportions*, Am. Stat. **52**, 119 (1998).

[67] L. D. Brown, T. T. Cai, and A. DasGupta, *Interval estimation for a binomial proportion*, Stat. Sci. **16**, 101 (2001).

[68] D. J. Williamson and T. J. Yoder, *Low-overhead fault-tolerant quantum computation by gauging logical operators*, arXiv:2410.02213.

[69] J. Ramette, J. Sinclair, N. P. Breuckmann, and V. Vuletić, *Fault-tolerant connection of error-corrected qubits with noisy links*, npj Quantum Inf. **10**, 58 (2024).

[70] H. Bombín, *Single-shot fault-tolerant quantum error correction*, Phys. Rev. X **5**, 031043 (2015).

[71] E. T. Campbell, *A theory of single-shot error correction for adversarial noise*, Quantum Sci. Technol. **4**, 025006 (2019).

[72] A. O. Quintavalle, M. Vasmer, J. Roffe, and E. T. Campbell, *Single-shot error correction of three-dimensional homological product codes*, PRX Quantum **2**, 020340 (2021).

[73] M. Ponce, R. van Zon, S. Northrup, D. Gruner, J. Chen, F. Ertinaz, A. Fedoseev, L. Groer, F. Mao, B. C. Mundim, M. Nolta, J. Pinto, M. Saldarriaga, V. Slavnic, E. Spence, C.-H. Yu, and W. R. Peltier, *Deploying a top-100 supercomputer for large parallel workloads: The Niagara supercomputer*, in *Proceedings of the Practice and Experience in Advanced Research Computing on Rise of the Machines Learning (PEARC'19)* (Association for Computing Machinery, New York, 2019).

[74] E. Swaroop, T. Jochym-O'Connor, and T. J. Yoder, *Universal adapters between quantum LDPC codes*, arXiv:2410.03628.